BASE24®

ERACOM Security Control Module (SCM) Reference Manual

ACI (Pacific) Pty. Ltd
1601 Malvern Road
Glen Iris VIC 3146
Australia
Telephone: (03) 9823-4500
Facsimile: (03) 9885-0766

Print Record

11/98, Version 1 (new)

Table of Contents

Applied Communications, Inc.

Preface

Reference Material

The following ERACOM documents are referred to:-

SECURITY CONTROL MODULE Ethernet Connect
300-39-00 Rev A4 3.93

The following BASE24 documents should be referred to:-

BASE24 TRANSACTION SECURITY
BASE24 LOGICAL NETWORK CONFIGURATION FILE
BASE24 FILES MANUAL

Applied Communications, Inc.

Section 1

Security Control Module Support Summary

The ERACOM Security Control Module Support facility provides the software components to support the ERACOM Security Control Module hardware. This facility provides encryption and key management required in the processing of EFT/POS, ATM and Switch transactions.

The ERACOM Security Control Module hardware is supported operating in:

- ethernet, binary mode.

The purpose of this document is to define the components of the ERACOM Security Control Module Support facility for BASE24-pos and BASE24-atm which will be referred to as BASE24.

It describes the functions of the components, how they interact, how they are controlled and the interfaces they have that are internal and external to the BASE24 products.

VISA support is an additional option requiring extra, compatible software components and a variation of ERACOM Security Control Module hardware.

The Security Module facility allows security device independence in the following BASE24 modules:

- Device Handler
- Authorisation
- Host Interface
- Switch Interface.

A uniform group of user-callable security utility procedures is provided for performing the following functions:

- Session Key generation
- Session Key translation
- Message Authentication Code generation
- Message Authentication Code verification
- Personal Identification Number translation
- Personal Identification Number verification
- Card Verification Value generation
- Card Verification Value verification.

The basic structure of the ERACOM Security Control Module support facility is similar to the ESM Security Module support.

Applied Communications, Inc.

Section 2

Glossary of Terms

Acronyms, Abbreviations, etc.

AFT	Application File Terminal
AST	Auth Selection Table
CAF	Cardholder Account File
CVV	Card Verification Value
EFT	Electronic Funds Transfer
ESM	Eracom Security Module
KVC	Key Verification Code
HCF	Host Configuration File
ICF	Interchange Configuration File
IDF	Institution Data File
MAC	Message Authentication Code
PIN	Personal Identification Number
POS	Point Of Sale
PSTM	POS Standard Internal Message
PTDF	POS Terminal Data File
SCM	Security Control Module
SPDH	Standard POS Device Handler module
STM	Standard Internal Message
TDF	Terminal Data File

Stored Keys

The SCM key management scheme complies with AS2805 Part 6, where only those keys which are highly sensitive or have long lives are stored within the SCM. All other keys are stored in the BASE24 database encrypted under the Domain Master Key or a variant thereof.

The following keys are stored in the ERACOM security module:-

KMs	Domain Master Keys (KMs) are double length keys which are used to encrypt keys and data stored in the BASE24 database. There are a maximum of sixteen KMs stored in the SCM at any one time.
KPVs	IBM3624 Pin Verification Keys (with their associated data). The SCM can verify PINs using the IBM3624 PIN Verification method. In order to accomplish this, PIN Verification Key for institution n; associated with Decimalisation Table for institution n, and the institution's minimal PIN length are all stored in the SCM (KPV is sometimes referred to as PVK).
KPVVA KPVVB	VISA PIN Verification Key Pairs. The SCM can verify PINs using the VISA PVV PIN verification method. This requires the storage of VISA PIN Verification Key Pairs (KPVVA/KPVVB) in the SCM.
KTKs	The SCM supports 16*KTKs. These can be used for a variety of purposes such as VISA Zone Control Master Keys.

Session Keys

Session keys (KS) are short-term keys which are randomly generated by the SCM, and shared by BASE24 and another network entity such as a terminal, a host or a switch.

There are 3 types of session keys which are used by BASE24:-

- PPK or PIN Protection Keys,
- MPK or MAC Processing Keys
- DPK or Data Protection Keys.

A PPK session key shared by BASE24 and an ATM is often called a communication key (KC). A PPK session key shared by BASE24 and an interchange is called a working key, and generally equates to one of two keys:-

- the Acquirer Working Key (AWK) or
- the Issuer Working Key (IWK).

Another general BASE24 term for PPK is PIN Encryption Key (KPE).

Key Variants

As with other security module types, the SCM uses different variants of a key encryption key to protect different types of keys.

Variants of keys are used to provide functional separation as detailed in AS2805 Part 6.1. The variants will be calculated as described in AS2805 Part 6.3 Clause 6.5 using the following constants:

Variant Number	Constant	Used to Protect
1	X'2424242424242424'	KMACs, KMAC
2	X'2828282828282828'	KPEs
3	X'2222222222222222'	KDs, KSD
4	X'4848484848484848'	KMACr
5	X'4242424242424242'	KPEr, KPP
6	X'4444444444444444'	KDr, KSK, KMPP
7	X'8282828282828282'	KEKs, KT, KRs, KZ
8	X'8484848484848484'	KEKr, KRr, KI, KTK
9	X'8888888888888888'	PPASN, KTI, MACRES
A	X'AAAAAAAAAAAAAAAA'	IBM3624 keys
*40	X'0606060606060606'	KeKs, KeKr

* This variant is in addition to standard variants. Implemented to support certain interchanges in which session keys are encrypted under the KEK and not a variant of the KEK.

Refer to Appendix A of Australian Standard 2805.6.1 document for description of Keys.

Key Generation

DEA Keys

The SCM has the facility to generate random single or double-length keys or key components. Such keys are all odd parity and specifically exclude those weak and semi-weak keys described in AS2805 Part 5.

RSA Keys

When an SCM is initialised there will be no RSA key pair available, but the SCM will immediately begin to generate a key pair in the background, storing it when generated.

Notation

E.X[Y] - The value "Y" encrypted in key "X" is represented in this notation. For example, if a session key (KS) was encrypted under the Domain Master Key (KM) the result would be represented as E.KM[KS].

The form of the encrypted value is usually represented to the user as a 16-hexadecimal-digit representation of eight bytes of encrypted data. In that form it is a "cryptogram" and can be passed as normal ASCII characters.

Applied Communications, Inc.

Section 3

Design Considerations

The SCM support facility interfaces to devices using ethernet communications. The bisync protocol is not supported.

SCM support is facilitated by a separate set of internal security utility procedures pertaining solely to SCM interfacing, which are transparent to these user modules.

To allow a BASE24 network to be configured to use an ERACOM device the following parameters and assigns need to be specified in the appropriate LCONF file.

Parameters:
- SECURE-DEV-TYP
- SECURE-DEV-ACCESS-TYPE
- SECURE-DEV-FAIL-NOTIFY
- SECURE-DEV-REINSTATE-TIME
- SECURE-DEV-TIM-LMT
- SECURE-DEV-TRN-RETRY-CNT
- SECURE-DEV-COMMS-TYPE

Assigns:
- SECURE-DEVn

See Section 12 for further information on configuring ERACOM devices.

Applied Communications, Inc.

Section 4

MODULE IMPACT SUMMARY

Wherever possible standard calls within HISWUTLS or SECUTILS code have been changed to interface to the appropriate SCM function call. Therefore, most BASE24 processes transparently call SCM unique utility procedures to perform required functions. The following briefly describes changes to standard BASE24 code.

The AFT system requires change to cater for the new management of keys in the ATM, POS and Switch environments. The files and screens effected are KEYA, KEYF, PTDF and TDF.

The Authorisation module transparently calls SCM unique utility procedures to verify PINs, and change PIN requests. It has been specifically changed to call the SCM function SCM^CARD^VERIFY to verify CVV/CVCs.

The SAA AS2805 Interface has been coded to call specific SCM functions for proof of endpoint processing, session key generation, and KVC processing. It transparently calls functions for PIN translation, MAC generation and verification.

The MDS Interface has been coded to call specific SCM function for key translates. It transparently calls functions for PIN translation.

The AS2805 Device Handler module has been coded to call specific SCM functions for terminal proof of endpoint processing and session key generation. It transparently calls functions for MAC generation and verification.

The Host/Switch Interfaces transparently call the PIN translation utility procedures unique to the SCM, depending on the LCONF PARAM SECURE-DEV-TYP. PINs from transaction requests are sent to the SCM to be translated using encryption under the Acquirer Working Key (AWK) before they are sent to the host/switch.

Applied Communications, Inc.

Section 5

Transaction Scenarios - PIN Processing

Transaction Originates at BASE24-pos

1. Transaction request is routed to the Device Handler module by XPNET. The PIN block is encrypted in a key which was sent to the terminal encrypted in the terminal base key (KBn) before the terminal was opened.

2. The Device Handler module determines from the PTDF fields ENCR-PIN.PIN-VAL-TERM ENCR-PIN.PIN-ENCRYPT-TYP and the LCONF PARAM SECURE-DEV-TYP that the terminal uses SCM key management and PIN security. Therefore it builds a PSTM that in addition to the normal data elements contains:

   (a) PIN-FRMT = PIN block format indicator:
   "0" = clear PIN
   "1" = ANSI PIN block encrypted
   "2" = IBM 3624 method (not used)
   "3" = PIN/pad format encrypted
   (b) PIN = E.KS[PIN block] (from terminal request usage)
   (c) PIN-KEY = E.KM1[KS] (from PTDF record)

3. The Router module receives the request and uses the AST to determine whether the request needs to be routed to the Authorisation module or to a Switch Interface process.

4. If the Router process passes the transaction request to the Authorisation module, then the latter determines whether the request needs to be authorised at a host, at BASE24, or at a switch.

5. If the request is to be authorised at a host:

   (a) The Host Interface process receives the request from the Authorisation module. The Host Interface process determines whether the host wants encrypted or clear PINs.
   (b) If the host requires encrypted PINs, the Host Interface process uses an SCM to translate the request PIN from encryption under the terminal's session key into encryption under the session key for the host.
   (c) If the host requires clear PINs, the Host Interface process uses an SCM to translate the PIN into encryption under the intermediate

key (KI) and then decrypts the PIN block into the clear, in software. Once in the clear, the Host Interface process performs any data formatting required to put the PIN block into the format expected by the host.

(d) If the Authorisation process has been configured to perform PIN verification then the PIN field will be blank and no PIN processing will be performed by the Host Interface.

(e) The Host Interface process sends the request to the host. Since the host either approves or denies the request, no further PIN processing is required.

6. If the request is to be authorised by BASE24-pos, the Authorisation module will use an SCM verification utility to format and send a request to an SCM to verify the encrypted PIN.

7. If the Router module passes the transaction request to a Switch Interface process, the Switch Interface process will use the SCM to translate the PIN block from encryption under the terminal's session key into encryption under the switch's session key. Once the PIN block is translated, the interface will send the encrypted PIN block to the switch.

Transaction Originates at BASE24-atm

1. Transaction request is routed to the Device Handler process by the XPNET Node. The PIN block is encrypted in a key which was sent to the terminal encrypted in the terminal base key (KBn) before the terminal was opened.

2. The Device Handler process determines from the TDF fields ENCRYPT-PIN.PIN-VAL-ATM ENCRYPT-PIN.PIN-ENCRYPT-TYP and the LCONF PARAM SECURE-DEV-TYP that the terminal uses SCM key management and PIN security. Therefore, it builds a STM that in addition to the normal data elements contains:

   (a) PIN-FRMT = PIN block format indicator:
   "0" = clear PIN
   "1" = ANSI PIN block encrypted
   "2" = IBM 3624 method (not used by BASE24)
   "3" = PIN/pad format encrypted
   (b) PIN = E.KS[PIN block] (from terminal request message)
   (c) PIN-KEY = E.KM1[S] (from TDF record).

3. Once the Authorisation process receives the request it determines whether the request needs to be authorised at a host, at BASE24, or at a switch.

4. If the request is to be authorised at a host:
   (a) The Host Interface receives the request from the Authorisation process. The Host Interface process determines whether the host wants encrypted or clear PINs.

(b) If the host requires encrypted PINs, the Host Interface process uses an SCM to translate the request PIN from encryption under the terminal's session key into encryption under the session key for the host.

(c) If the host requires clear PINs, the Host Interface process uses an SCM to translate the PIN into encryption under the intermediate key (KI) and then decrypts the PIN block into the clear, using software. Once in the clear the Host Interface process performs any data formatting required to put the PIN block into the format expected by the host.

(d) If the Authorisation process has been configured to perform PIN verification then the PIN field will be blank and no PIN processing will be performed by the Host Interface process.

(e) The Host Interface process sends the request to the host. Since the host either approves or denies the request, no further PIN processing is required.

5. If the request is to be authorised by BASE24-atm, the Authorisation process will use an SCM verification utility to format and send a request to an SCM to verify the encrypted PIN.

6. If the Authorisation process passes the transaction request to a Switch Interface process, the Switch Interface process will use the SCM to translate the PIN block from encryption under the terminal's session key into encryption under the switch's session (working) key. Once the PIN block is translated, the interface process will send the encrypted PIN block to the switch.

Transaction Originates at Host Computer or Switch

1. Transaction request arrives at the Interface process.

2. The Interface process determines whether the host/switch provides clear or encrypted PINs from information in the KEYF file.

3. If the host/switch sends clear PINs, the Interface process will change the PIN block format if necessary and then send a request to the SCM to encrypt the PIN block under the Issuer Working Key. The IWK is stored encrypted under KM1 in the KEYF file. Once encrypted, the Interface builds the PSTM/STM with:

   PIN-FRMT = PIN block format indicator
   PIN = E.IWK[PIN block] - encrypted by the Interface
   PIN-KEY = E.KM1[[WK] (INBOUND KEYF data)

4. If the host/switch sends encrypted PINs, the Interface process builds the PSTM/STM and adds:
   PIN-FRMT = PIN block format indicator
   PIN = E.IWK[PIN block] - PIN block encrypted in the Issuer
      Working Key (from the host/switch)
   PIN-KEY = E.KM1[IWK] (INBOUND KEYF data)

5. The request is sent to the Router module (ie BASE24-pos). The Router module will determine whether to send it to the Authorisation module or whether to send it to a Switch Interface process for routing to a switch.

6. If the Authorisation module verifies the request, it will take data from the PSTM/STM and its data files and send a PIN verify request to the SCM. The SCM will reply indicating whether the PIN had been verified.

7. If the Authorisation module sends the request to a host, the Host Interface process will translate the PIN into the host's session key (AWK ie OUTBOUND of KEYF) via an SCM before forwarding the request to the host (see the previous scenario explanation).

8. If the request is routed to a Switch Interface process, the interface process will use an SCM to translate the PIN from encryption under the origin host's/switch's session key (IWK ie INBOUND of KEYF) into encryption under the switch's session key (AWK ie OUTBOUND of KEYF).

Section 6

ATM Terminal Data File Screens

The ATM Terminal Data File (TDF) defines the characteristics relating to an ATM terminal.

All terminal keys (master and session) must be stored in the TDF. The SCM does not store any terminal keys.

To indicate that the terminal uses the SCM, the following field in the TDF needs to be set up:

PIN ENCRYPTION TYPE -> "03".

The SCM implementation within BASE24 supports four different types of ATM, i.e. Diebold, IBM 36XX, IBM 4730 and NCR 50XX devices.

DIEBOLD

The security screen for a Diebold ATM is as follows:

```
BASE24-ATM   TERMINAL DATA            ____  ____  __/__/__  __:__  09 OF 12
 TERM ID:  ________________   FIID: ____  REGION: ____  BRANCH: ____  LN: ____

                      DIEBOLD SECURITY INFORMATION

       MASTER   KM ID: __    KEY:  ________________   CHECK DIGITS: ______

                             PIN INFORMATION

       PIN ENCRYPT TYPE:  __   (______________________________)
PIN CHANGE ENCRYPT TYPE:  __   (______________)
       LOCAL PIN VERIFY:  _    (______________________________)


                    MESSAGE AUTHENTICATION INFORMATION

       MAC SUPPORT TYPE:  _    (_______________)
          MAC DATA TYPE:  _    (_____)


 *********************************** BASE24 ***********************************
  NEW PAGE:        FILE DESTINATION:        NEW LOGICAL NETWORK ID:
                       F12-HELP
```

Fields displayed on screen:

DIEBOLD.ENCRYPT-PIN.ENCRYPT-KEYS.M-KEY
Specifies the master key for this terminal, stored in binary format using 9 bytes (includes the KM ID).

DIEBOLD.ENCRYPT-PIN.ENCRYPT-KEYS.M-KEY-CHK-VALUE
Specifies the check digits for the master key, stored in binary format using 3 bytes

DIEBOLD.ENCRYPT-PIN.PIN-ENCRYPT-TYP
The type of PIN encryption the terminal employs to keep the PIN secure when transmitting it across the data communications line to the Tandem.
Values are:

- 00 = No encryption. PINs are to be transmitted from the terminal in the clear.
- 01 = DES algorithm (ANSI standard PIN/PAD; without a security device). PINs are to be transmitted from the terminal in encrypted PIN/PAD format and decrypted by BASE24 via the DES algorithm.
- 03 = DES algorithm (ANSI standard PIN/PAD; with a security device). PINs are to be transmitted from the terminal in encrypted PIN/PAD format and under keys that are only in the clear within a security module.
- 04 = DES algorithm (ANSI standard PIN/PAN; without a security device). PINs are to be transmitted from the terminal in encrypted ANSI standard PIN/PAN format and decrypted by BASE24 via the DES algorithm.
- 05 = DES algorithm (ANSI standard PIN/PAN; with a security device). PINs are to be transmitted from the terminal in encrypted ANSI standard PIN/PAN format and under keys that are only in the clear within a security module.

DIEBOLD.PIN-CHG-ENCRYPT-TYP
A code used with a PIN change transaction indicating the type of new PIN encryption the terminal employs to secure the new PIN data when transmitting it across the data communications line to the Tandem.
Values are:

- 00 = No encryption. PINs are to be transmitted from the terminal in the clear.
- 01 = DES algorithm (ANSI standard PIN/PAD; without a security device). PINs are to be transmitted from the terminal in encrypted PIN/PAD format and decrypted by BASE24 via the DES algorithm.
- 03 = DES algorithm (ANSI standard PIN/PAD; with a security device). PINs are to be transmitted from the terminal in encrypted PIN/PAD format and under keys that are only in the clear within a security module.
- 04 = DES algorithm (ANSI standard PIN/PAN; without a security device). PINs are to be transmitted from the terminal in encrypted ANSI standard PIN/PAN format and decrypted by BASE24 via the DES algorithm.
- 05 = DES algorithm (ANSI standard PIN/PAN; with a security device). PINs are to be transmitted from the terminal in encrypted ANSI standard PIN/PAN format and under keys that are only in the clear within a security module.

Note that the allowed values in this field depend on the value in the PIN-ENCRYPT-TYP field. The following table shows the valid values for this field based on the PIN-ENCRYPT-TYP field:

PIN-ENCRYPT-TYP	PIN-CHG-ENCRYPT-TYP
00	00
01	00 or 01
03	00 or 03
04	00 or 04
05	00 or 05

DIEBOLD.LOCL-PIN-VRFY-FLG
A code indicating when the ATM device can perform PIN verification without the involvement of the BASE24 network. The ATM device must have PIN verification capability to perform local PIN verification, and the ATM configuration file must be configured for local PIN verification. The value in this field is used as input to a fraud prevention check for financial and administrative transactions whose PIN has been verified locally by the terminal.

Valid values are:

0 = Allow local PIN verification
1 = Do not allow local PIN verification
2 = Allow local PIN verification for on-us only

DIEBOLD.MAC.MAC-TYP
The type of message authentication that is used by the Device Handler process.

SCM support level 7.1 does not support MAC'ing.

The valid values are:

0 = No Message Authentication
1 = Software Message Authentication
3 = Hardware Message Authentication

DIEBOLD.MAC.MAC-DATA-TYP
The character set that the data to be authenticated will be in.

SCM support level 7.1 does not support MAC'ing.

The valid values are:

0 = ASCII
1 = EBCDIC

Fields not displayed on screen:

DIEBOLD.ENCRYPT-PIN.ENCRYPT-KEYS.P-KEY
An encrypted version of the ATM's communication key. The clear version of this key is the ATM communication key. The clear communication key is encrypted using the clear Master Key and is loaded to the ATM in an encrypted format.

DIEBOLD.MAC.KEY-VAL
The MAC key that is used in the authentication of a specified message. If a hardware security device is being used, this key is encrypted under a key known by the security module. If MAC processing is being performed in software, this key represents the clear MAC key generated via software.

SCM support level 7.1 does not support MAC'ing.

IBM 36XX

The security screen for an IBM 36XX ATM is as follows:

```
BASE24-ATM   TERMINAL DATA            ____  FI01  __/__/__  __:__  09 OF 12
 TERM ID:  ________________   FIID: ____  REGION: ____  BRANCH: ____ LN: ____

                   IBM 3624 SECURITY INFORMATION

       PIN ENCRYPT TYPE:  __  (______________________________)
         MASTER   KM ID:  __  KEY:  ________________  CHECK DIGITS: ______

PIN CHANGE ENCRYPT TYPE:  __  (______________)
       LOCAL PIN VERIFY:  _   (______________________________)









 *********************************** BASE24 ***********************************
  NEW PAGE:        FILE DESTINATION:        NEW LOGICAL NETWORK ID:
                       F12-HELP
```

Fields displayed on screen:

I36XX.ENCRYPT-PIN.PIN-ENCRYPT-TYP
The type of PIN encryption the terminal employs to keep the PIN secure when transmitting it across the data communications line to the Tandem.
Values are:

00 = No encryption. PINs are to be transmitted from the terminal in the clear.
01 = DES algorithm (ANSI standard PIN/PAD; without a security device). PINs are to be transmitted from the terminal in encrypted PIN/PAD format and decrypted by BASE24 via the DES algorithm.
03 = DES algorithm (ANSI standard PIN/PAD; with a security device). PINs are to be transmitted from the terminal in encrypted PIN/PAD

format and under keys that are only in the clear within a security module.

I36XX.ENCRYPT-PIN.ENCRYPT-KEYS.M-KEY
Specifies the master key for this terminal, stored in binary format using 9 bytes (includes the KM ID) has been redefined as follows I36XX.ENCRYPT-PIN.ENCRYPT-KEYS.SCM.E-KMV10-M (not referenced).

I36XX.ENCRYPT-PIN.ENCRYPT-KEYS.M-KEY-CHK-VALUE
Specifies the check digits for the master key, stored in binary format using 3 bytes has been redefined as follows I36XX.ENCRYPT-PIN.ENCRYPT-KEYS.M-KVC (not referenced).

I36XX.PIN-CHG-ENCRYPT-TYP
A code used with a PIN change transaction indicating the type of new PIN encryption the terminal employs to secure the new PIN data when transmitting it across the data communications line to the Tandem. The only valid value for the IBM 3624 device type is:

00 = No encryption.
PINs are to be transmitted from the terminal in the clear.

I36XX.LOCL-PIN-VRFY-FLG
A code indicating when the ATM device can perform PIN verification without the involvement of the BASE24 network. The ATM device must have PIN verification capability to perform local PIN verification and the ATM configuration file must be configured for local PIN verification. The value in this field is used as input to a fraud prevention check for financial and administrative transactions whose PIN has been verified locally by the terminal.
Valid values are:

0 = Allow local PIN verification
1 = Do not allow local PIN verification
2 = Allow local PIN verification for on-us only

Fields not displayed on screen:

I36XX.ENCRYPT-PIN.ENCRYPT-KEYS.A-KEY
Specifies the new PIN encryption key for this terminal only 9 bytes used

I36XX.ENCRYPT-PIN.ENCRYPT-KEYS.P-KEY
Specifies the current PIN encryption key for this terminal only 9 bytes used

IBM 4730

The security screen for an IBM 4730 ATM is as follows:

```
BASE24-ATM   TERMINAL DATA            ____  ____  __/__/__  __:__  09 OF 12
 TERM ID:  _______________   FIID: ____  REGION: ____  BRANCH: ____  LN: ____

                   IBM 4730 SECURITY INFORMATION

       PIN ENCRYPT TYPE:  __  (_____________________________)
         MASTER   KM ID:  __  KEY:  ________________  CHECK DIGITS: ______

PIN CHANGE ENCRYPT TYPE:  __  (_____________________________)
       LOCAL PIN VERIFY:  _   (_____________________________)










 *********************************** BASE24 ***********************************
  NEW PAGE:        FILE DESTINATION:        NEW LOGICAL NETWORK ID:
                       F12-HELP
```

Fields displayed on screen:

I4730.ENCRYPT-PIN.PIN-ENCRYPT-TYP
The type of new PIN encryption the terminal employs to secure the PIN when transmitting it across the data communications line to the Tandem.
Values are:

01 = DES algorithm (ANSI standard PIN/PAD; without a security device). PINs are to be transmitted from the terminal in encrypted PIN/PAD format and decrypted by BASE24 via the DES algorithm.

03 = DES algorithm (ANSI standard PIN/PAD; with a security device). PINs are to be transmitted from the terminal in encrypted PIN/PAD format and under keys that are only in the clear within a security module.

I4730.ENCRYPT-PIN.ENCRYPT-KEYS.M-KEY
Specifies the master key for this terminal, stored in binary format using 9 bytes (includes the KM ID).

I4730.ENCRYPT-PIN.ENCRYPT-KEYS.M-KEY-CHK-VALUE
Specifies the check digits for the master key, stored in binary format using 3 bytes

I4730.PIN-CHG-ENCRYPT-TYP
A code used with a PIN change transaction indicating the type of new PIN encryption the terminal employs to secure the new PIN data when transmitting it across the data communications line to the Tandem. Values are:

01 = DES algorithm (ANSI standard PIN/PAD; without a security device). PINs are to be transmitted from the terminal in encrypted PIN/PAD format and decrypted by BASE24 via the DES algorithm.

03 = DES algorithm (ANSI standard PIN/PAD; with a security device). PINs are to be transmitted from the terminal in encrypted PIN/PAD format and under keys that are only in the clear within a security module.

Note that the allowed values in this field depend on the value in the PIN-ENCRYPT-TYP field. The following table shows the valid values for this field based on the value contained in the PIN-ENCRYPT-TYP field:

PIN-ENCRYPT-TYP		PIN-CHG-ENCRYPT-TYP
01	01	00
03	03	

I4730.LOCL-PIN-VRFY-FLG
A code indicating when the ATM device can perform PIN verification without the involvement of the BASE24 network. The ATM device must have PIN verification capability to perform local PIN verification and the ATM configuration file must be configured for local PIN verification. The value in this field is used as input to a fraud prevention check for financial and administrative transactions whose PIN has been verified locally by the terminal. Valid values are:

0 = Allow local PIN verification
1 = Do not allow local PIN verification
2 = Allow local PIN verification for on-us only

Fields not displayed on screen:

I4730.ENCRYPT-PIN.ENCRYPT-KEYS.A-KEY
Specifies the new PIN encryption key for this terminal only 9 bytes used

I4730.ENCRYPT-PIN.ENCRYPT-KEYS.P-KEY
Specifies the current PIN encryption key for this terminal only 9 bytes used

NCR 50XX

The security screen for a NCR 50XX ATM is as follows:

```
BASE24-ATM   TERMINAL DATA            ____  ____  __/__/__  __:__  09 OF 12
 TERM ID:  ________________   FIID: ____  REGION: ___  BRANCH: ____  LN: ____

                  NCR 5070/5080 SECURITY INFORMATION

        MASTER KM ID : __   KEY: ________________   CHECK DIGITS: ______

                            PIN INFORMATION

        PIN ENCRYPT TYPE:  __  (______________________________)
 PIN CHANGE ENCRYPT TYPE:  __  (______________)
        LOCAL PIN VERIFY:  _   (______________________________)

                  MESSAGE AUTHENTICATION INFORMATION

        MAC SUPPORT TYPE:  _  (______________)
           MAC DATA TYPE:  _  (_____)
    SOLICITED STATUS MSG:  _  (____________)
      CALCULATION METHOD:  _  (_______________________________)

 *********************************** BASE24 ***********************************
  NEW PAGE:        FILE DESTINATION:        NEW LOGICAL NETWORK ID:
                      F12-HELP
```

Fields displayed on screen:

N5080.ENCRYPT-PIN.ENCRYPT-KEYS.M-KEY
Specifies the master key for this terminal, stored in binary format using 9 bytes (includes the KM ID).

N5080.ENCRYPT-PIN.ENCRYPT-KEYS.M-KEY-CHK-VALUE
Specifies the check digits for the master key, stored in binary format using 3 bytes

N5080.ENCRYPT-PIN.PIN-ENCRYPT-TYP
The type of PIN encryption the terminal employs to keep the PIN secure when transmitting it across the data communications line to the Tandem. Values are:

- 00 = No encryption. PINs are to be transmitted from the terminal in the clear.
- 01 = DES algorithm (ANSI standard PIN/PAD; without a security device). PINs are to be transmitted from the terminal in encrypted PIN/PAD format and decrypted by BASE24 via the DES algorithm.
- 03 = DES algorithm (ANSI standard PIN/PAD; with a security device). PINs are to be transmitted from the terminal in encrypted PIN/PAD format and under keys that are only in the clear within a security module.
- 04 = DES algorithm (ANSI standard PIN/PAN; without a security device). PINs are to be transmitted from the terminal in encrypted ANSI standard PIN/PAN format and decrypted by BASE24 via the DES algorithm.
- 05 = DES algorithm (ANSI standard PIN/PAN; with a security device). PINs are to be transmitted from the terminal in encrypted ANSI standard PIN/PAN format and under keys that are only in the clear within a security module.

N5080.PIN-CHG-ENCRYPT-TYP
A code used with a PIN change transaction indicating the type of new PIN encryption the terminal employs to secure the new PIN data when transmitting it across the data communications line to the Tandem. Values are:

00 = No encryption. PINs are to be transmitted from the terminal in the clear.
01 = DES algorithm (ANSI standard PIN/PAD; without a security device). PINs are to be transmitted from the terminal in encrypted PIN/PAD format and decrypted by BASE24 via the DES algorithm.
03 = DES algorithm (ANSI standard PIN/PAD; with a security device). PINs are to be transmitted from the terminal in encrypted PIN/PAD format and under keys that are only in the clear within a security module.
04 = DES algorithm (ANSI standard PIN/PAN; without a security device). PINs are to be transmitted from the terminal in encrypted ANSI standard PIN/PAN format and decrypted by BASE24 via the DES algorithm.
05 = DES algorithm (ANSI standard PIN/PAN; with a security device). PINs are to be transmitted from the terminal in encrypted ANSI standard PIN/PAN format and under keys that are only in the clear within a security module.

Note that the allowed values in this field depend on the value in the PIN-ENCRYPT-TYP field. The following table shows the valid values for this field based on the PIN-ENCRYPT-TYP field:

PIN-ENCRYPT-TYP	PIN-CHG-ENCRYPT-TYP
00	00
01	00 or 01
03	00 or 03
04	00 or 04
05	00 or 05

N5080.LOCL-PIN-VRFY-FLG
A code indicating when the ATM device can perform PIN verification without the involvement of the BASE24 network. The ATM device must have PIN verification capability to perform local PIN verification and the ATM configuration file must be configured for local PIN verification. The value in this field is used as input to a fraud prevention check for financial and administrative transactions whose PIN has been verified locally by the terminal. Valid values are:

0 = Allow local PIN verification
1 = Do not allow local PIN verification
2 = Allow local PIN verification for on-us only

N5080.MAC.MAC-TYP
The type of message authentication that is used by the Device Handler process.

SCM support level 7.1 does not support MAC'ing.

The valid values are:
0 = No Message Authentication
1 = Software Message Authentication
3 = Hardware Message Authentication

N5080.MAC.MAC-DATA-TYP
The character set that the data to be authenticated will be in.

SCM support level 7.1 does not support MAC'ing.

The valid values are:
0 = ASCII
1 = EBCDIC

Fields not displayed on screen:

N5080.ENCRYPT-PIN.ENCRYPT-KEYS.P-KEY
An encrypted version of the ATM's communication key. The clear version of this key is the ATM communication key. The clear communication key is encrypted using the clear Master Key and is loaded to the ATM in an encrypted format.

N5080.MAC.KEY-VAL
The MAC key that is used in the authentication of a specified message. If a hardware security device is being used, this key is encrypted under a key known by the security module. If MAC processing is being performed in software, this key represents the clear MAC key generated via software.

SCM support level 7.1 does not support MAC'ing.

Section 7

POS Terminal Data File Screens

The POS Terminal Data File (PTDF) defines the characteristics relating to a POS terminal.

All terminal keys (master and session) must be stored in the PTDF.
The SCM does not store any terminal keys.

To indicate that the terminal uses the SCM, the following fields in the PTDF need to be set up:

VALIDATE MAC	-> "Y"
VALIDATE PIN	-> "Y"
MAC TYPE	-> "91" AS2805 6.2 Software
	-> "92" AS2805 6.4 Software
	-> "93" AS2805 6.2 Hardware
	-> "94" AS2805 6.4 Hardware
PIN ENCRYPTION	-> "91" AS2805 6.2 Software
TYPE	-> "92" AS2805 6.4 Software
	-> "93" AS2805 6.2 Hardware
	-> "94" AS2805 6.4 Hardware

Note: The only BASE24 POS Device Handler module to support these values is the AS2805 Device Handler.

The security screen for the PTDF is as follows:

```
BASE24-POS   TERMINAL DATA           ____        __/__/__  __:__  07 OF 18

 TERMINAL ID:  ________________                        FIID: ____

                          ENCRYPTION KEYS

     PIN PAD CHARACTER:  _

   PIN ENCRYPTION TYPE:  __  (NO ENCRYPTION)
        PIN MASTER KEY:  ________________       CHECK DIGITS:  ____
          VALIDATE PIN:  _


              MAC TYPE:  __  (__________)
        MAC MASTER KEY:  ________________       CHECK DIGITS:  ____




 *********************************** BASE24 ***********************************
  NEW PAGE:        FILE DESTINATION:       NEW LOGICAL NETWORK ID:
                          F12-HELP
```

See BASE24 BASE FILE MAINTENANCE MANUAL for a more detailed description.

For the AS2805 DH the following fields have been added for AS2805 6.4 support.

Fields not displayed on screen:

PTDF^DDD.SESS.CURR.E^KMV1^KMACs
Specifies current MAC Sending Key

PTDF^DDD.SESS.CURR.E^KMV4^KMACr
Specifies current MAC Receiving Key

PTDF^DDD.SESS.CURR.E^KMV5^KPPr
Specifies current PIN Encryption Key

PTDF^DDD.SESS.CURR.E^KMV3^KDs
Specifies current Data Sending Key not used by AS2805 Device Handler module

PTDF^DDD.SESS.CURR.E^KMV6^KDr
Specifies current Data Receiving Key not used by AS2805Device Handler module

PTDF^DDD.SESS.CURR.E^KMV7^KEK1
Specifies current Key Exchange Key 1

PTDF^DDD.SESS.CURR.E^KMV7^KEK2
Specifies current Key Exchange Key 2

PTDF^DDD.SESS.PREV
Specifies previous version of above keys

PTDF^DDD.E^KMV9^PPASN
Specifies PIN PAD Acquirer Serial Number (PPASN)

PTDF^DDD.KIA
Specifies Acquirer Initialisation Key (KIA) currently not used by the AS2805 Device Handler.

Section 8

Interface Screen Field Usage

KEYF Impacts

The Key File (KEYF) contains information required for the secure management of PINs and authentication of messages in a BASE24 system by interface processes.

The KEYF file contains one record for each Interface/Interchange process associated with the system. Information on the record contains specific key processing information.

Screen 1 - PIN encryption/translation and MAC keys

```
BASE24-BASE  KEY FILE                  ____  ____  __/__/__  __:__  01 OF 04
         DPC/FIID: ____         INTERFACE PROCESS: ________________

    ENCRYPT TYPE: _ (______________)    BASE24 ENCRYPT TYPE: _ (______________)
PIN BLOCK FORMAT: _ (____)                  ANSI PAN FORMAT: _ (________________)
MAC ENCRYPT TYPE: _ (______________)      PIN PAD CHARACTER: _
   MAC DATA TYPE: _ (_____)                  NUMBER OF KEYS: _ (_______)
      KEY LENGTH: _ (______)               FULL MESSAGE MAC: _ (________________)

                                 INTERMEDIATE KEYS
          CLEAR: ________________                      CHECK DIGITS: ____
      ENCRYPTED: ________________

                                   EXCHANGE KEYS
        PIN KEY: ________________  ________________   CHECK DIGITS: ____
        MAC KEY: ________________  ________________   CHECK DIGITS: ____
        KEKs: __ ________________  ________________   KVC: ______
        KEKr: __ ________________  ________________   KVC: ______

 RECORD LAST CHANGED: __/__/__  __:__   BY USER: ____ , ________   ADD
 *********************************** BASE24 ***********************************
  NEW PAGE:        FILE DESTINATION:        NEW LOGICAL NETWORK ID:
                        F12-HELP
```

Refer BASE24 BASE FILE MAINTENANCE MANUAL for a more detailed description.

Fields displayed on screen:

PRIKEY-HCF.DPC-NUM / PRIKEY-ICF.FIID
The DPC number of the host or FIID of the Interchange using this record.

PRIKEY-HCF.HISF-PRO / PRIKEY-ICF.SWI-PRO
The name of the Interface process associated with the DPC or Interchange.

INTERFACE.ENCRYPT-TYP
Specifies how PINs are handled by DPC or Interchange.

INTERFACE.B24-ENCRYPT-TYP
Specifies how PINs are handled internally by BASE24.

INTERFACE.PIN-BLK
Specifies the PIN block format of inbound and outbound PINS.

INTERFACE.ANSI-PAN
Specifies the PAN digits used by the DPC or Interchange in forming ANSI PIN blocks.

INTERFACE.MAC-TYP
Specifies how MAC'ing will be handled between the DPC or Interchange and the BASE24 Interface process.

INTERFACE.PINPAD-CHAR
Specifies the pad character used by the DPC or Interchange in forming PIN/PAD PIN blocks.

INTERFACE.MAC-DATA-TYP
Specifies the character set used for message transmission between DPC or Interchange and the BASE24 Interface process. (Not used by SAA).

INTERFACE.NUM-KEYS
Specifies whether inbound and outbound keys are combined or separate. (Not used by SCM).

INTERFACE.KEY-LGTH
Specifies whether single- or double-length Key Exchange Keys are being used by Interface. (Not used by SCM).

INTERFACE.FULL-MSG-MAC
Specifies whether selected data items or the entire message will be used to create the MAC. (Full message MAC'ing is always used by SAA).

INTERFACE.INTERM.KEY-CLEAR
The clear version of the Intermediate Key. (Not used by SCM).

INTERFACE.INTERM.KEY-CHK-VALUES
Specifies the check digits corresponding to the value in the Encrypted Intermediate Key. (Not used by SCM).

INTERFACE.INTERM.KEY-ENCRYPT
Specifies the security module Encrypted Intermediate Key. (Not used by SCM).

INTERFACE.EXCHNG-KEY (-EXTND)
Specifies the security module encrypted form of the key used to exchange PIN keys (single and double length). (Not used by SCM).

INTERFACE.EXCHNG-KEY-CHK-VALUES
Specifies the check digits corresponding to the value in the PIN Key field. (Not used by SCM).

INTERFACE.MAC-EXCHNG-KEY (-EXTND)
Specifies the security module encrypted form of the key used to exchange MAC keys (single and double length). (Not used by SCM).

INTERFACE.MAC-EXCHNG-KEY-CHK-VALUES
Specifies the check digits corresponding to the value in the MAC Key field. (Not used by SCM support).

INTERFACE.E^KMV7^KEKS
Specifies the security module encrypted form of the key used to encrypt the PIN and MAC keys sent. (Used specifically by SCM)

INTERFACE.E^KMV8^KEKR
Specifies the security module encrypted form of the key used to decrypt the PIN and MAC keys received. (Used specifically by SCM)

Screen 2 PIN/MAC inbound/outbound working keys

```
BASE24-BASE  KEY FILE                 ____  ____  __/__/__  __:__  02 OF 04
         DPC/FIID: ____          INTERFACE PROCESS: ________________

                               PIN KEY INFORMATION
OUTBOUND KEYS:
   PIN KEY1: ________________    CHECK DIGITS1: ____    CURRENT INDEX: _
   PIN KEY2: ________________    CHECK DIGITS2: ____      KEY COUNTER: ______
INBOUND KEYS:
   PIN KEY1: ________________    CHECK DIGITS1: ____    CURRENT INDEX: _
   PIN KEY2: ________________    CHECK DIGITS2: ____      KEY COUNTER: ______

                               MAC KEY INFORMATION
OUTBOUND KEYS:
   MAC KEY1: ________________    CHECK DIGITS1: ____    CURRENT INDEX: _
   MAC KEY2: ________________    CHECK DIGITS2: ____      KEY COUNTER: ______
INBOUND KEYS:
   MAC KEY1: ________________    CHECK DIGITS1: ____    CURRENT INDEX: _
   MAC KEY2: ________________    CHECK DIGITS2: ____      KEY COUNTER: ______

 RECORD LAST CHANGED: __/__/__  __:__   BY USER: ____ , ________   ADD
 *********************************** BASE24 ***********************************
  NEW PAGE:       FILE DESTINATION:       NEW LOGICAL NETWORK ID:
                        F12-HELP
```

ReferBASE24 BASE FILE MAINTENANCE MANUAL for a more detailed description.

This screen is NOT used by SCM support.

INTERFACE.OUTBOUND.PIN.KEY1
A key used to encrypt (or translate) PINs sent to DPC or Interchange from BASE24.

INTERFACE.OUTBOUND.PIN.KEY-CHK-VALUE1
The check digits corresponding to the value of the PIN KEY1 field.

INTERFACE.OUTBOUND.PIN.CURR-INDX
Specifies which outbound PIN encryption key is currently being used by the Interface.

INTERFACE.OUTBOUND.PIN.KEY2
A key used to encrypt (or translate) PINs sent to DPC or Interchange from BASE24.

INTERFACE.OUTBOUND.PIN.KEY-CHK-VALUE2
The check digits corresponding to the value of the PIN KEY2 field.

INTERFACE.OUTBOUND.PIN.KEY-CNTR
Specifies the number of times the outbound PIN key has been changed by dynamic key management processing.

INTERFACE.INBOUND.PIN.KEY1
A key used to encrypt PINs sent to BASE24 from the DPC or Interchange.

INTERFACE.INBOUND.PIN.KEY-CHK-VALUE1
The check digits corresponding to the value of the PIN KEY1 field.

INTERFACE.INBOUND.PIN.CURR-INDX
Specifies which inbound PIN encryption key is currently being used by the Interface.

INTERFACE.INBOUND.PIN.KEY2
A key used to encrypt PINs sent to BASE24 from the DPC or Interchange.

INTERFACE.INBOUND.PIN.KEY-CHK-VALUE2
The check digits corresponding to the value of the PIN KEY2 field.

INTERFACE.INBOUND.PIN.KEY-CNTR
Specifies the number of times the inbound PIN key has been changed by dynamic key management processing.

INTERFACE.OUTBOUND.MAC.KEY1
A key used to generate MACs sent to DPC or Interchange from BASE24.

INTERFACE.OUTBOUND.MAC.KEY-CHK-VALUE1
The check digits corresponding to the value of the MAC KEY1 field.

INTERFACE.OUTBOUND.MAC.CURR-INDX
Specifies which outbound MAC key is currently being used by the Interface.

INTERFACE.OUTBOUND.MAC.KEY2
A key used to generate MACs sent to DPC or Interchange from BASE24.

INTERFACE.OUTBOUND.MAC.KEY-CHK-VALUE2
The check digits corresponding to the value of the MAC KEY2 field.

INTERFACE.OUTBOUND.MAC.KEY-CNTR
Specifies the number of times the outbound MAC key has been changed by dynamic key management processing.

INTERFACE.INBOUND.MAC.KEY1
A key used to generate MACs sent to BASE24 from the DPC or Interchange.

INTERFACE.INBOUND.MAC.KEY-CHK-VALUE1
The check digits corresponding to the value of the MAC KEY1 field.

INTERFACE.INBOUND.MAC.CURR-INDX
Specifies which inbound MAC key is currently being used by the Interface.

INTERFACE.INBOUND.MAC.KEY2
A key used to generate MACs sent to BASE24 from the DPC or Interchange.

INTERFACE.INBOUND.MAC.KEY-CHK-VALUE2
The check digits corresponding to the value of the MAC KEY2 field.

INTERFACE.INBOUND.MAC.KEY-CNTR
Specifies the number of times the inbound MAC key has been changed by dynamic key management processing.

Screen 3 Dynamic Key management

```
BASE24-BASE  KEY FILE                  ____  ____  __/__/__  __:__  03 OF 04
         DPC/FIID: ____          INTERFACE PROCESS: _________________

         PIN KEY VARIANT: 0                         MAC KEY VARIANT: _
                                      LIMITS
       PIN KEY TIMER VALUE:    _                MAC KEY TIMER VALUE:    _
    PIN KEY TIMER INTERVAL: _ (________)    MAC KEY TIMER INTERVAL: _ (________)
              PIN KEY TRAN:      _                     MAC KEY TRAN:      _
             PIN KEY ERROR:    _                      MAC KEY ERROR:    _
 CONSECUTIVE PIN KEY ERROR:    _          CONSECUTIVE MAC KEY ERROR:    _
                                         KMAC SYNCHRONIZATION ERROR:    _

 CLEAR OLD KEY TIMER VALUE:    _ (________)

            ORIGINATING ID:
              RECEIVING ID:

       KEY PROCESSING TYPE: _ (____)
    NOTARIZATION SUPPORTED: _ (______________)
 RECORD LAST CHANGED: __/__/__  __:__   BY USER: ____ , ________   ADD
*********************************** BASE24 ***********************************
  NEW PAGE:        FILE DESTINATION:        NEW LOGICAL NETWORK ID:
                         F12-HELP
```

See BASE24 BASE FILE MAINTENANCE MANUAL for a more detailed description.

This screen is NOT used by SCM support.

INTERFACE.PIN-KEY-VARIANT
Specifies which key variant to be applied to the PIN Key Exchange Key (KEK) when translating PIN keys.

INTERFACE.MAC-KEY-VARIANT
Specifies which key variant to be applied to the MAC Key Exchange Key (KEK) when translating MAC keys.

INTERFACE.PIN-KEY-TIMER-LMT
Specifies the maximum length of time a PIN key should be used for.

INTERFACE.MAC-KEY-TIMER-LMT
Specifies the maximum length of time a MAC key should be used for.

INTERFACE.PIN-KEY-TRAN-LMT
Specifies the maximum number of transactions that can be performed with the current PIN key, before changing the key.

INTERFACE.MAC-KEY-TRAN-LMT
Specifies the maximum number of transactions that can be performed with the current MAC key, before changing the key.

INTERFACE.PIN-ERR-LMT
Specifies the maximum number of PIN key related translation errors that can occur with the current PIN key before changing the key.

INTERFACE.MAC-ERR-LMT
Specifies the maximum number of MAC verification errors that can occur with the current MAC key before changing the key.

INTERFACE.CONS-PIN-ERR-LMT
Specifies the maximum number of consecutive PIN key related translation errors that can occur with the current PIN key before changing the key.

INTERFACE.CONS-MAC-ERR-LMT
Specifies the maximum number of consecutive MAC verification errors that can occur with the current MAC key before changing the key.

INTERFACE.KMAC-SYNC-ERR-LMT
Specifies the maximum number of key synchronisation errors that can occur with the present MAC key before changing the key.

INTERFACE.OLD-KEY-TIMER-LMT
Specifies the maximum length of time (in seconds) that an old key can be used before it is automatically cleared by the Interface process.

INTERFACE.ORG-ID
Specifies the sender of Dynamic Key Exchange messages.

INTERFACE-RCV-ID
Specifies the receiver to Dynamic Key Exchange messages.

INTERFACE.KEY-PROC-TYP
Specifies the type of Dynamic Key Management processing this Interface can perform.

INTERFACE.NOTARIZE-FLG
Specifies whether this Interface supports Key Notarisation.

Note: KEYF screen 3 is not used by SAA for SCM support. Instead, a new screen on the ICF (screen 12) is used to provide Dynamic Key Management - see Section 8.2.

Screen 4 SCM Static Key management

```
BASE24-BASE  KEY FILE                  ____        __/__/__  __:__  04 OF 04
         DPC/FIID: ____          INTERFACE PROCESS: ________________

                               SCM STATIC KEY SUPPORT

OUTBOUND KEYS:

   PIN KEY1 __ ________________  PIN KEY2 __ ________________  CURRENT INDEX: _
   MAC KEY1 __ ________________  MAC KEY2 __ ________________  CURRENT INDEX: _


INBOUND KEYS:

   PIN KEY1 __ ________________  PIN KEY2 __ ________________  CURRENT INDEX: _
   MAC KEY1 __ ________________  MAC KEY2 __ ________________  CURRENT INDEX: _




 RECORD LAST CHANGED: __/__/__  __:__   BY USER: ____ , ________   CHANGE
 ********************************* BASE24 ***********************************
  NEW PAGE:        FILE DESTINATION:        NEW LOGICAL NETWORK ID:
                         F12-HELP
```

Fields displayed on this screen:

INTERFACE.SCM.OUTBOUND.PIN.SKEY1
A key used to encrypt (or translate) PINs sent to DPC or Interchange from BASE24.

INTERFACE.SCM.OUTBOUND.PIN.SKEY2
A key used to encrypt (or translate) PINs sent to DPC or Interchange from BASE24.

INTERFACE.OUTBOUND.PIN.CURR-INDX
Specifies which PIN encryption key is currently being used.

INTERFACE.SCM.OUTBOUND.MAC.SKEY1
A key used to encrypt (or translate) MACs sent to DPC or Interchange from BASE24.

INTERFACE.SCM.OUTBOUND.MAC-SKEY2
A key used to encrypt (or translate) MACs sent to DPC or Interchange from BASE24.

INTERFACE.OUTBOUND.PIN.CURR-INDX
Specifies which MAC encryption key is currently being used.

INTERFACE.SCM.INBOUND.PIN.SKEY1
A key used to encrypt PINs sent to BASE24 from the DPC or Interchange.

INTERFACE.SCM.INBOUND.PIN.SKEY2
A key used to encrypt PINs sent to BASE24 from the DPC or Interchange.

INTERFACE.INBOUND.PIN.CURR-INDX
Specifies which inbound PIN encryption key is currently being used.

INTERFACE.SCM.INBOUND.MAC.SKEY1
A key used to generate MACs sent to BASE24 from the DPC or Interchange.

INTERFACE.SCM.INBOUND.MAC.SKEY2
A key used to generate MACs sent to BASE24 from the DPC or Interchange.

INTERFACE.INBOUND.MAC.CURR-INDX
Specifies which inbound MAC key is currently being used.

ICF Impacts

Screen 12 Key Parameters

```
BASE24-BASE  ICF FILE MAINTENANCE     QUA1  FI01  98/09/21  14:34  12 OF 13
INTERCHANGE FIID: SAA                   PROCESS: P1A^SAA4

                          ICF KEY PARAMETERS

 ATM SETTLE TIME: 23:59  POS SETTLE TIME: 23:59  STAND IN ALLOWED: N (Y/N)
  CUTOVER METHOD: 0 (SEND/RECEIVE 0520)  RECON FLAG: 0 (BOTH ATM AND POS)
 ATM DEFAULT ACC: 00 DEFAULT             POS DEFAULT ACC: 00 DEFAULT

                      SESSION KEY ROLLOVER THRESHOLDS
          KEY EXCHANGE BAD PINS: 336       BAD MACS:  336
   NUM TRANS FOR TOTALS REWRITE: 336       NUM TRANS KEY EXCHANGE: 2336
               SAF TIME UPDATE: N (Y/N)   CALC MAC ON REPEAT: N (Y/N)

        KEY EXCHANGE INFORMATION                              INCLUDE
 TYPE OF KEY EXCHANGE: 3 (DYNAMIC)                     DATA KEY      : N (Y/N)
 TAPE FILE FOR KEYS:                                   KVC PROCESSING: Y (Y/N)
 TAPE DATE:        (YYMMDD)                            PROOF ENDPOINT: N (Y/N)


 *********************************** BASE24 ***********************************
  NEW PAGE:        FILE DESTINATION:        NEW LOGICAL NETWORK ID:
                              F12-HELP
```

Refer BASE24 BASE FILE MAINTENANCE MANUAL for a more detailed description.

Fields displayed on screen:

ATM-SETL-TIM.ATM-SETL-HH / .ATM-SETL-MM
Specifies the hour and minute that the SAA Interchange process should begin ATM reconciliation with its partner.

POS-SETL-TIM.POS-SETL-HH / .POS-SETL-MM
Specifies the hour and minute that the SAA Interchange process should begin POS reconciliation with its partner.

STAND-IN-AUTH
Specifies whether stand-in processing is allowed for the interchange.

CUTOVER-METHOD
Specifies the cutover method for the interchange (0 = send/recv 0520's; 1 = send 0520 only; 2 = receive 0520 only; 3 = no 0520 exchanged).

RECONCILIATION-FLG
Specifies which totals will be sent to the interchange partner at reconciliation and how many messages will be sent.

DEFAULT-ACC-ATM
Specified the default ATM account when a request is received from an interchange partner.

DEFAULT-ACC-POS
Specifies the default POS account when a request is received from an interchange partner.

TRANS-COUNT-OPTIONS.NUM-BAD-KEYS-PIN
Specifies the number of bad PIN verifications that are to be allowed before initiating a Key Exchange.

TRANS-COUNT-OPTIONS.NUM-BAD-KEYS-MAC
Specifies the number of bad MACs that are to be allowed before initiating a Key Exchange.

TRANS-COUNT-OPTIONS.NUM-TRANS-ICF-REWRITE
Specifies the number of transactions that the SAA Interchange process will process before updating the first record (ie. totals record) in the current ILF.

NUM-TRANS-KEY-EXCHANGE
Specifies the number of transactions that the SAA Interchange process will process before initiating a Key Exchange.

SAF-TIM-UPD
Specifies whether the transmit time is updated before sending a SAF transaction to the co-network.

MAC-REPEAT-CALC
Specifies whether the MAC is calculated on the repeat or original transaction when sending a repeat SAF transaction.

KEY-EXCHNG-TYP
Specifies how SAA is to perform its key management (Static, Dynamic or Tape).

INCL-DATA-KEY
Specifies whether the Data Key should be included in the transaction.

TKEY-FILE-NAME
Specifies the name of the Tape File containing the PIN and MAC keys. Only used if the Key Exchange Type field is set to 2 (TAPE KEY FILE).

INCL-KVC
Specifies whether KVC processing is supported by this Interchange.

TAPE-DATE
Specifies the date of the Tape File. This date is compared with the value on the Tape File to ensure the correct file is being read. Only used if the Key Exchange Type is set to 2 (TAPE KEY FILE).

PROOF-ENDPOINT
Specifies whether proof of endpoint is supported by the Interchange.

Section 9

Institution Screen Field Usage

KEYA Impacts

The KEYA file contains information required by the Authorisation module for verifying PINs and cards. (Note: PIN verification is controlled via IDF or CPF, Card verification is controlled by CPF only).

Screen 1 requests the information required to select the appropriate KEYA record.

Screen 2 DES (IBM 3624) PIN verification

```
BASE24-BASE  KEY AUTH FILE            ____  ____  __/__/__  __:__  02 OF 06

  GRP: ____ BEGIN DATE (YYYYMM): ****** END DATE (YYYYMM): ****** FIID: ____



                    DES (IBM 3624) PIN VERIFICATION

  DECIMALIZATION TABLE: ________________         PAN VERIFY OFFSET: __
             CLEAR KEY: ________________         PAN VERIFY LENGTH: __
         ENCRYPTED KEY: ________________         PAN PAD CHARACTER: _
          CHECK DIGITS: ____






 RECORD LAST CHANGED: __/__/__  __:__   BY USER: ____ , ________   CHANGE
 ********************************** BASE24
***********************************
  NEW PAGE:        FILE DESTINATION:        NEW LOGICAL NETWORK ID:
                    F12-HELP
```

Refer BASE24 BASE FILE MAINTENANCE MANUAL for a more detailed description.

Fields displayed on screen:

IBM-DES.DEC-TBL
Not used as originally specified - however, it specifies the number of digits in PIN block to check. If not specified a default value of ‘4’ is used by the SCM routines.

IBM-DES.PAN-VFY-OFST
Specifies the first position of the card number (PAN) to be used in the formation of the validation data used in DES (IBM 3624) PIN verification.

IBM-DES.KEY-CLEAR
Not used as PIN Verification keys are stored in SCM.

IBM-DES.PAN-VFY-LGHT
Specifies the number of positions of the card number (PAN) to be used in the formation of the validation data used in DES (IBM 3624) PIN verification.

IBM-DES.KEY-ENCRYPT
Not used as originally specified, it specifies the institution index under which the PVK and decimalisation table are stored in the SCM (the index must be entered in hex, right-justified, with zero-fill on the left).

IBM-DES.PAN-PAD
Specifies the character used to pad validation data used in DES (IBM 3624) PIN verification.

VERIFY.IBM-DES.ENCRYPT-CHK-VALUES
Specifies the check digit corresponding to the value in the Encrypted Key field. (Not used by SCM).

The PIN verification key is not stored in the KEYA for SCMs, but rather in the SCM itself.

For DES (IBM 3624) PIN verification the PIN DATA fields are set up for use in the PIN^VERIFY^IBM^DES user utility.

The field PAN1 which is supplied to the SCM for PIN verification is built by the Authorisation process using IBM-DES.PAN-VFY-OFST, IBM-DES.PAN-VFY-LGHT and IBM-DES.PAN-PAD.

Screen 5 VISA PVV PIN verification

```
BASE24-BASE  KEY AUTH FILE            ____  ____  __/__/__  __:__  05 OF 06

  GRP: ____ BEGIN DATE (YYYYMM): ****** END DATE (YYYYMM): ****** FIID: ____

                            VISA PVV KEYS
         CLEAR            ENCRYPT                 CLEAR            ENCRYPT
  1 ________________  ________________    ________________  ________________
       CHECK DIGITS:  ____                   CHECK DIGITS:  ____
  2 ________________  ________________    ________________  ________________
       CHECK DIGITS:  ____                   CHECK DIGITS:  ____
  3 ________________  ________________    ________________  ________________
       CHECK DIGITS:  ____                   CHECK DIGITS:  ____
  4 ________________  ________________    ________________  ________________
       CHECK DIGITS:  ____                   CHECK DIGITS:  ____
  5 ________________  ________________    ________________  ________________
       CHECK DIGITS:  ____                   CHECK DIGITS:  ____
  6 ________________  ________________    ________________  ________________
       CHECK DIGITS:  ____                   CHECK DIGITS:  ____


 *********************************** BASE24 ***********************************
  NEW PAGE:        FILE DESTINATION:       NEW LOGICAL NETWORK ID:
                    F12-HELP
```

Refer BASE24 BASE FILE MAINTENANCE MANUAL for a more detailed description.

Fields displayed on screen:

ABA-VISA.KEY-CLEAR
Not used as VISA PIN Verification Key pairs are stored in the SCM.

ABA-VISA.KEY-ENCRYPT
Not used as originally specified - however, it specifies the index under which the VISA PIN Verification Key pair is stored in the SCM (the index must be entered in hex, right-justified, with zero-fill on the left) - this index must be entered for the appropriate PVKi.

VERIFY.ABA-VISA.KEY-ENCRYPT-CHK-VALUES
Specifies the check digits corresponding to the value in the Encrypt field (above this field on screen). Not used by the SCM.

Screen 6 Card Verification

```
BASE24-BASE  KEY AUTH FILE            ____  ____  __/__/__  __:__  06 OF 06

  GRP: ____ BEGIN DATE (YYYYMM): ****** END DATE (YYYYMM): ****** FIID: ____


                            CARD VERIFICATION

        ENCRYPT TYPE: __ (__________)

            CLEAR KEY             ENCRYPTED KEY      CHECK DIGITS
      1  ________________        ________________       ____
      2  ________________        ________________       ____

      KCVV: __   ________________________________    KVC: ______
      CVV LENGTH:  _



 *********************************** BASE24 ***********************************
  NEW PAGE:        FILE DESTINATION:        NEW LOGICAL NETWORK ID:
                    F12-HELP
```

Refer BASE24 BASE FILE MAINTENANCE MANUAL for a more detailed description.

Fields displayed on screen:

VERIFY.CV.ENCRYPT-TYP
Not used as VISA PIN Verification Key pairs are stored in the SCM.

VERIFY.CV.KEY-CLEAR
Specifies the clear CVK pair to be used when Card Verification is performed in software.

VERIFY.CV.KEY-ENCRYPT
Specifies the security module encrypted CVK pair to be used when Card Verification is performed by hardware. This field should be set “01” to indicate use of encrypted keys.

VERIFY.CV.KEY-ENCRYPT-CHK-VALUES
The check digits corresponding to the value in each Encrypted Key field (next to this field on screen).

VERIFY.CV.E-KMV4-KCVV
The SCM encrypted form of Card Verification Key pair as a double length key.

VERIFY.CV.KCVV-KVC
The SCM KVC of Card Verification Key.

VERIFY.CV.CVV-LGTH
The number of CVV digits to check, default is 3 (used by SCM).

CPF Impacts

Screen 2 Card Verification

```
BASE24-BASE    CARD PREFIX            ____       __/__/__  __:__  02 OF 22
 PREFIX: ___________         PAN LENGTH: __                FIID: ____

                                    PIN INFORMATION
    PIN VERIFICATION KEYA GROUP: ____      PIN CHECK TYPE: __ ___ (________)

                  MAX PIN TRIES: ___       BAD PIN ACTION: _  (____________)
          CARDHOLDER PIN SELECT: _ (Y/N)  ALGO NUMBER LOC: _  (____________)
       CHECK IF HOST ONLINE PIN: _ (Y/N)    POFST/PVV LOC: _  (____)
  PIN TRIES RESET OPTION: _ (__________)

                            CARD VERIFICATION INFORMATION
   CARD VERIFICATION KEYA GROUP: ____        CV CHECK TYPE: _ (_______)
                        CV DATE: ____  CHECK IF HOST ONLINE CV: _ (Y/N)
                TRACK1 CVD OFST:  _    TRACK1 SRVC CODE OFST:  _
                TRACK2 CVD OFST: __    TRACK2 SRVC CODE OFST: __

          BAD CV ACTION - TRACK DATA COMPLETE:  _ (_____________)
          BAD CV ACTION - TRACK DATA UNCERTAIN: _ (_____________)
 RECORD LAST CHANGED: __/__/__  __:__    BY USER: ____ , ________   CHANGE
 *********************************** BASE24 ***********************************
  NEW PAGE:        FILE DESTINATION:        NEW LOGICAL NETWORK ID:
                     F12-HELP
```

Refer BASE24 BASE FILE MAINTENANCE MANUAL for a more detailed description.
(Note: Field descriptions for PIN Information are not included here).

Fields displayed on screen:

CPFBASE.CV-KEYA-GRP
The value used by the Authorisation module to select the correct KEYA record when performing Card Verification for cards with this prefix.

CPFBASE.CV-CHK-TYP
Specifies when card verification is to be attempted when processing complete and uncertain Track 2 data.

CPFBASE.CV-EFF-DAT
Contains the CVV expiration date. If a card had an expiration date greater than or equal to this date, a CVV value is expected on the card and CVV processing will

occur. Otherwise, the card may not have a CVV value and CVV processing will not occur.

CPFBASE.CV-CHK
Specifies whether BASE24 should perform Card Verification during transaction screening prior to sending a transaction to the Host.

CPFBASE.TRK1-CV-OFST
Specifies the position of the Card Verification Digits within the card's Track1 data.

CPFBASE.TRK1-SC-OFST
Specifies the position of the Service Code within the card's Track1 data.

CPFBASE.CV-OFST
Specifies the position of the Card Verification Digits within the card's Track2 data.

CPFBASE.SC-OFST
Specifies the position of the Service Code within the card's Track2 data.

CPFBASE.CV-BAD-DISP[0]
Specifies the action to be taken when complete Track2 data is present and the card's CVD fails to be verified.

CPFBASE.CV-BAD-DISP[1]
Specifies the action to be taken when the condition of the card's Track2 data is uncertain and the card's CVD fails to be verified.

Applied Communications, Inc.

Section 10

POS Terminal Data File (PTDF) Screen Field Usage

The ENCR-PIN.ENCR-KEYS.M-KEY field of the PTDF is used to store an index to the key referred to as the terminal BASE KEY (KB), in ERACOM nomenclature. It matches the index entered at the ERACOM console just prior to entering the terminal base key and is a number from 1 to 99.

More than 99 terminals may be supported with the same BASE24-pos Device Handler module by using the same base key for multiple terminals.

The index number entered in the ENCR-PIN.ENCR-KEYS.M-KEY field of the PTDF is right-justified and zero-filled by the AFT operator.

To indicate that the terminal uses SCM security for PIN processing the following fields are required:-

- LCONF PARAM SECURE-DEV-TYP is set to "SCM"
- ENCR-PIN.PIN-ENCRYPT-TYP of PTDF is set to "03" or "04"
- ENCR-PIN.PIN-VAL-TERM is set to "0".

To indicate that that the terminal uses SCM security for MAC processing the following fields are required:-

- LCONF PARAM SECURE-DEV-TYP is set to "SCM"
- MAC-DATA.MAC-VAL-TERM is set to "0"
- MAC-DATA.MAC-ENCRYPT-TYP is set to "03"

The MAC-DATA.ENCR-KEYS.MAS-KEY field of the PTDF contains the terminal index number, as for PIN processing (see above).

Applied Communications, Inc.

Section 11

ATM Terminal Data File (TDF) Screen Field Usage

The ENCRYPT-PIN.ENCRYPT-KEYS.M-KEY field of the TDF is used to store an index to the key referred to as the terminal BASE KEY (KB), in ERACOM nomenclature. It matches the index entered at the ERACOM console just prior to entering the terminal base key and is a number from 1 to 99.

More than 99 terminals may be supported with the same BASE24-atm Device Handler process, by using the same base key for multiple terminals.

The index number entered in the ENCRYPT-PIN.ENCRYPT-KEYS.M-KEY field of the TDF is right-justified and zero-filled by the AFT operator.

To indicate that the terminal uses SCM security for PIN processing the following fields are required:-

- LCONF PARAM SECURE-DEV-TYP is set to "SCM"
- ENCRYPT-PIN.PIN-ENCRYPT-TYP of TDF is set to "03"

Applied Communications, Inc.

Section 12

Security Device Assignment and Configuration.

LCONF

During initialisation, the application process retrieves its symbolic name and the assigns and parameters directed to it. The PIN^INIT utility is used to retrieve the device names of the SCMs from the LCONF file.

Params

The parameter, SECURE-DEV-TYP, enables the application process to determine the type of security control module that is used for PIN management and authentication by BASE24. SECURE-DEV-TYP must be set to "SCM" to indicate that a Security Control Module (Level 07.10) device is used.

The parameter, SECURE-DEV-FAIL-NOTIFY, defines the interval, in seconds, that messages are logged for denied transactions due to security device failures (default 20).

The parameter, SECURE-DEV-REINSTATE-TIME, defines the time, in minutes, after which a "DOWNED" security device is to be brought "UP" (default 0).

The parameter, SECURE-DEV-TIM-LMT, defines the number of seconds to wait for a response from a security device before a transaction is considered to be timed out (default 5).

The parameter, SECURE-DEV-TRN-RETRY-CNT, defines the number of tries per transaction (mac generation, PIN verification, etc.) before rejection (default 5).

Assigns

The assign, SECURE-DEV# defines a particular security device (where # may be from 1 to 5). For a SCM which uses the Ethernet communications protocol, this assign refers to a Blocker process. The Blocker process is described in Appendix E of this document.

Note The parameter, SECURE-DEV-ACCESS-TYPE is not used by SCM support i.e. it only supports round-robin processing in a multi-device environment

See the *LCONF manual, BASE24 Logical Network Configuration File Manual* for further details.

Error Handling

The SCM devices are OPENed and any errors are reported when the routine PINI^CONVERSE^WITH^DEVICES is invoked from the PIN^INIT user utility routine.

SCM functions are tried on a round robin basis when more than one SCM device is configured in the LCONF.

If a key management function fails because the SCM has a hardware error or fails to respond, the routine PINI^CRITICAL^ERROR, invoked from the application process by one of the user utility routines, logs the problem. The particular SCM is then marked as unavailable, and the operation is retried with the next available SCM. These actions are performed internal to the user utility routines.

The application process continues to send requests on a round robin basis excluding the downed device until it receives a warmboot command. On receiving a warmboot command, the Device Handler module closes and reopens all Blocker processes by calling the PIN^INIT user utility routine, and if successful, the downed SCM is marked as available.

Section 13

POS AS2805 Device Handler Modules

A POS Device Handler module is used to control transactions that come from a group of POS terminals. Each POS device is supported by a POS Terminal Data File (PTDF) which defines the characteristics relating to a particular POS device.

The AS2805 Device Handler module is the only POS Device Handler module that currently supports SCM.

AS2805 Device Handler SCM

Initialisation

AS2805 6.4 - non RSA

When initialising a POS device, there are several keys which must be first entered into the PIN Pad. The SCM "Function Code 3130" will randomly generate the necessary keys for the PIN Pad and the PTDF. The keys generated for the PIN Pad are encrypted under a transport key and for the PTDF encrypted under variants of KM.

The keys destined for the PTDF are entered into a new file, the Keyset file (KSF). This file contains all the key sets generated by the SCM for the Pin Pad population. When PIN Pads are injected with a set of keys the KSF is updated with the PIN Pad Identification (PPID) to indicate which PIN Pad a particular set of keys belong.

The KeySet Lookup file (KSLF) manages the linking of PIN Pads with the appropriate key set in the KSF.

On a logon request the Device Handler module compares the PPID in the logon request message from the device with the PTDF^DDD.SESS.PPID. If these values do not match then the current key set will be retrieved from the KSF by the Device Handler module.

In the logon request, the left hand 32 bits of the PIN Pad Security Number (PPSN)(encrypted under KEK) is received from the terminal. The SCM "Function Code E000" compares this value with the PPSN stored in the PTDF (encrypted under KMV9). If SCM verification is positive, Proof-of-Endpoint has been established in direction of terminal to acquirer.

Using an SCM "Function Code 3000", allows the generation of session keys, (i.e. a Send MAC Key, a Send Data Key, a Receive MAC Key, a Receive PIN Protection Key and a Receive Data Key). These keys are sent to the POS device encrypted under the new Key Encrypting Key 1 (KEK1). They are also stored in the PTDF (encrypted under the appropriate variant of KM).

If KEK1 has become inoperable, then the session keys must be generated based on Key Encrypting Key 2 (KEK2). Using an SCM "Function Code 3010", new session keys, a new KEK1 and a new KEK2 will be generated. The new session keys are sent to the POS device encrypted under the new KEK1. They are also stored in the PTDF (encrypted under the appropriate variant of KM).

AS2805 6.4 - RSA

NB This facility is currently not implemented.

Message Authentication

AS2805 6.4

Message Authentication Code (MAC) processing is performed if the PTDF field MAC-DATA.MAC-ENCRYPT-TYP is set to "94".

On receipt of a financial message from the POS device, the Device Handler module first must verify that the incoming MAC is correct. The Device Handler module uses the PIN^MAC^VERIFY utility, with the encrypted MAC PTDF^DDD.SESS.CURR.E^KMV4^KMACr (from the PTDF) and calls the SCM "Function Code 5100". If the MAC is not correct, the message is tried with the previous MAC key PTDF^DDD.SESS.PREV.E^KMV4^KMACr (from the PTDF) and calls the SCM "Function Code 5100". If the MAC is not correct, the message is rejected. If it is correct then the PREV keys are copied to the CURR keys and processing continues.

For each financial message that is transmitted to the POS device, the Device Handler module must calculate a MAC and append it to the end of the message. The Device Handler module uses the PIN^MAC^GENERATE utility, with the encrypted MAC key PTDF^DDD.SESS.CURR.E^KMV1^KMACs (from the PTDF) and calls the SCM "Function Code 5000".

Financial Transactions

AS2805 6.4

When a financial request is received from the POS device, the Device Handler module must format the PSTM with the required fields for the Authorisation module to be able to verify the PIN. The following PSTM fields must be loaded:

PSTM.PIN	->	Encrypted PIN Block
PSTM.PIN-SIZE	->	"16"
PSTM.PIN-FRMT	->	"9" (AS2805 6.4 Pin Block Format)
PSTM.PIN-KEY	->	PTDF^DDD.SESS.CURR.E^KMV5^KPPr
PSTM.PINPAD-CHAR	->	PIN-PAD-CHAR from PTDF
PSTM.ANSI-OFST	->	calculated from PAN

Applied Communications, Inc.

Section 14

ATM Device Handler Modules

An ATM Device Handler process is used to control transactions that come from a group of ATM terminals. Each ATM is supported by a Terminal Data File (TDF) which defines the characteristics relating to the particular ATM.

To indicate that the terminal utilises SCM support, the following field in the TDF needs to be set:

ENCR-PIN.PIN-ENCRYPT-TYP -> "03"

BASE24 supports four different types of ATM, namely Diebold, IBM 36XX, IBM 4730 and NCR 50XX devices.

Diebold and NCR 50XX ATMS

Initialisation of ATM

To initialise the ATM, the A-Key must be created by an SCM "Function Code 3600" and entered into the ATM and the TDF (encrypted under KMV10). This should be performed via a stand-alone utility.

The B key which is entered into the ATM is not placed in the TDF as BASE24 never requires the "base" PIN session key to be used.

BASE24 will always generate a new PIN session key which is sent to the ATM.

PIN Session Key Change

On receipt of a load request from the ATM, the Device Handler process uses an SCM "Function Code 3620" to generate a new PIN session key. The new PIN session Key is returned as three cryptograms, (i.e. encrypted under the A-Key to be sent back to the terminal on a load response, encrypted under KMV5 for storage on the TDF and encrypted under KMV10 to be used as a new PIN Master Key). BASE24 does not use the last cryptogram.

Financial Transactions

When a financial request is received from the ATM, the Device Handler process must format the STM with the required fields for the Authorisation process to verify the PIN. The following STM fields must be loaded:

STM.RQST.PIN	->	Encrypted PIN Block
STM.RQST.PIN-SIZE	->	"16"
STM.RQST.PIN-FRMT	->	"1" for ANSI, "3" for PIN/PAD
STM.RQST.PIN-KEY	->	PIN session key binary data only 9 bytes used
STM.RQST.PINPAD-CHAR	->	"F"
STM.RQST.ANSI-OFST	->	0

MAC support

The BASE24 ATM Device Handler modules support MAC processing via a hardware security module. However SCM Level 7.1 does not provide the necessary functions eg the generation of MAC session keys encrypted under variants 1 and 4 based on the A key i.e e*KMV10(A).

IBM 36XX

Initialisation of ATM

A-Key

To initialise the ATM, the A-Key must be created by an SCM "Function Code 3600" and entered into the ATM and the TDF. This is performed via a stand-alone utility, SCMCOM.

B-Key

The standard BASE24 functionality is to provide this support in software with the use of a hard coded B Key which would be used by all ATM's connected to the Device Handler process. The Device Handler process would use this B Key as the BASE C Key. In hardware the SCM function '3600' described above does not provide the necessary variants of the B Key i.e variant 3 and variant 6 and so currently BASE24 cannot support the use of hardware support for the communications key. See Appendix B, 3600 ABKEYGEN.

Master Key Change

On receipt of a load request from the ATM, the Device Handler process uses an SCM "Function Code 3620" to generate a Master Key. The Master Key is returned as three cryptograms (i.e. encrypted under the A-Key to be sent back to the terminal on a load response, encrypted under KMV5 for storage on the TDF and encrypted under KMV10). The KMV10 variant of the Master Key is not stored on the TDF.

Communications Key Change

When it becomes necessary to generate a new Communications Key for the ATM, the Device Handler process uses a software procedure to generate a new Communication Key. The new Communications Key is returned as a cryptogram (i.e. encrypted under the old communications Key to be sent back to the terminal on a load response and in the clear for storage on the TDF).

Financial Transactions

When a financial request is received from the ATM, bytes 4 to 11 must first be deciphered by the Communications Key in software.
The Device Handler process then must format the STM with the required fields for the Authorisation process to verify the PIN. The following STM fields must be loaded:

STM.RQST.PIN	->	Encrypted PIN Block
STM.RQST.PIN-SIZE	->	"16"
STM.RQST.PIN-FRMT	->	"1" for ANSI, "3" for PIN/PAD
STM.RQST.PIN-KEY	->	Master Key binary data only 9 bytes
STM.RQST.PINPAD-CHAR	->	"F"
STM.RQST.ANSI-OFST	->	0

On response to the ATM, bytes 4 to 11 will be enciphered by the Communications Key in software.

IBM 4730 ATMS

Initialisation of ATM

A-Key

To initialise the ATM, the A-Key must be created by an SCM "Function Code 3600" and entered into the ATM and the TDF. This is performed via a stand-alone utility, SCMCOM.

Exchange Key

The standard BASE24 functionality is to provide this support in software with the use of a hard coded Exchange Key which would be used by all ATM's connected to the DH.

Master Key Change

On receipt of a load request from the ATM, the Device Handler process uses an SCM "Function Code 3620" to generate a Master Key. The Master Key is returned as three cryptograms (i.e. encrypted under the A-Key to be sent back to the terminal on a load response, encrypted under KMV5 for storage on the TDF and encrypted under KMV10). The KMV10 variant of the Master Key is not stored on the TDF.

Communications Key Change

When it becomes necessary to generate a new Communications Key for the ATM, the Device Handler process uses a software procedure to generate a new Communications Key. The new Communications Key is returned as a cryptogram (i.e. encrypted under the Exchange Key to be sent back to the terminal on a load response and in the clear for storage on the TDF).

Financial Transactions

When a financial request is received from the ATM, the MAC is verified and the PIN block is decrypted by the Communications Key in software. (The PIN block is doubly encrypted with Communications Key and PIN session key.)

The Device Handler process then must format the STM with the required fields for the Authorisation process to verify the PIN. The following STM fields must be loaded:

STM.RQST.PIN	->	Encrypted PIN Block
STM.RQST.PIN-SIZE	->	16"
STM.RQST.PIN-FRMT	->	”1" for ANSI, "3" for PIN/PAD
STM.RQST.PIN-KEY	->	PIN session Key binary data only 9 bytes
STM.RQST.PINPAD-CHAR	->	"F"
STM.RQST.ANSI-OFST	->	0

On response to the ATM, the MAC is generated using the Communications Key in software.

MAC support

The BASE24 ATM Device Handler process supports MAC processing in software only and uses the current Communications Key as the MAC session key. See Appendix B.

Section 15

SAA Interface Process

The SAA Interface makes extensive use of the SCM in supporting the AS2805 standards.

Currently, the only security scheme supported by the SAA and SCM support is AS2805 6.3 (i.e double length Key Encrypting Key KEKs, session keys encrypted under variants of KEKs).

Pre-initialisation of Interface

Session keys are used over an interface for message authentication and the encryption of data and PINs. There will be a set of session keys for each direction where each set must be encrypted by a Key Encrypting Key (KEK).

The KEYF for the SAA Interchange process needs to have *KEKs and *KEKr entered so that the SAA Interchange process can perform the necessary functions.

Manual Exchange of KEKs and Public Keys

Through the use of SCMCOM the user will be able to get access to the Public key and have it written to disc to be sent to the interchange partner. Refer to SCMCOM function GPB.

Through the use of SCMCOM the user will be able to generate a *KEKs in the formats described in Appendix C. A version of the *KEKs is stored on the KEYF and the other is written to disc to be sent the interchange partner. Refer to SCMCOM function KSE.

SCMCOM will also accept a partner's KEK (i.e *KEKr) and, using the function described in Appendix C, will translate the key into the required format so that it can entered into the KEYF file. Refer to SCMCOM function KSR.

Manual Exchange of test KEK keys

Test KEKs - Key Encrypting send Key

- decide on test *KEKs eg 11111111111111112222222222222222

- inform partner of *KEKs in clear

- using SCMCOM to generate e*KMV7(*KEKs) by having KM 0 loaded into SCMCOM through the KEYFILE option.

Refer to Tandem files BA51SCM.GOSCMCOM and BA51SCM.KEYFILE.

Note: The default KM's for SCMCOM are:

- 0 - 00112233445566778899AABBCCDDEEFF
- 1 - 112233445566778899AABBCCDDEEFF00)

Go to software functions within SCMCOM, namely CRM followed by DDD (i.e double length encrypt).

- specify variant 7 and enter key *KEKs
- note result and place in KEYF, screen 1, KEKs

Test KEKr - Key Encrypting receive Key

- get test *KEKr eg 22222222222222221111111111111111 from partner
- using SCMCOM to generate e*KMV8(*KEKr)
- go to software functions within SCMCOM namely CRM followed by DDD (i.e double length encrypt)
- specify variant 8 and enter key *KEKr
- note result and place in KEYF, screen 1, KEKr

Manual Exchange of production KEK keys

First exchange Public keys

- using SCMCOM function GPB, write GPB to disc (eg KPS) and transfer to diskette
- deliver diskette to partner.

To accept partner's Public key, transfer from diskette to disc (eg KPR).

Production KEKs - Key Encrypting Send Key

- using SCMCOM function KSE to generate random KEKs, will require access to disc file KPR, note details displayed as well as written to diskette and KEYF. Refer to Appendix B, C100 NODEKEKGEN for a description of SCM function call.

The encrypted *KEKs and KVC are written to disc, transfer to diskette and delivered to partner.

User’s KEYF should be updated by SCMCOM.

Production KEKr - Key Encrypting Receive Key

- accept diskette from partner and place on disc (eg BANKKEKR)
- using SCMCOM function KSR to accept random KEKr file (eg BANKKEKR), will also require access to partners public previously exchanged (eg KPR), note details displayed and write KEYF. Refer to Appendix B, C110 NODEKEKTRANS for a description of SCM function call.

Online Exchange of Public Key

Note: This facility is currently not implemented.

Online Exchange of KEKs

To implement this facility set the field “Type of Key Exchange” on Screen 12 of the ICF file to “3”.

The interchange process will read the KEYF file to retrieve the KPR, when required through a new command to SAA NODEKEKSEND. To generate a Send Key Encrypting Key (KEKs) the SAA must use an SCM "Function Code C100". Translating a Receive Key Encrypting Key from the remote node requires an SCM "Function Code C110".

Proof-of-Endpoint

Before any transactions can take place, each interface node must perform a Proof-of-Endpoint function.

An SCM "Function Code E020" generates a random key which must be forwarded to the remote node. On receipt of the random key, the remote node uses an SCM "Function Code E030" to invert the random key.

This inverted random key is passed back to the original node where a comparison is made against a stored version of the inverted random key. If the keys are equivalent, then proof-of-endpoint has been established in one direction.

If Proof of Endpoint is not supported by your partner then through an ICF setting processing can be inhibited.

Session Key Change

Using an SCM "Function Code 3200", allows the generation of session keys, (i.e. a Send MAC Key, a Send PIN Encryption Key and a Send Data Key). These keys are sent to the remote node encrypted under the Send Key Encrypting Key. These keys are only stored in the PCT table (i.e if the SAA Interchange process is stopped then we will be forced to exchange keys as part of the logon process).

Upon receipt of new session keys from the remote node, an SCM "Function Code 4000" must be utilised to translate the keys into a recognisable form. This function will reveal the Receive MAC Key, the Receive PIN Encryption Key and the Receive Data Key.

As part of receiving keys the SAA Interchange process will generate the KVC for each key through the use of "Function Code 7000" and place them in the "0830" response to the partner.
When the SAA Interchange process receives the KVCs it will verify them against the KVCs in memory. If not equal it will force a logoff and start the logon processing again.

If Data keys or KVC processing are not supported by your partner then through an ICF setting (Screen 12), processing can be inhibited.

Message Authentication

For each financial message that is transmitted across the interface, a Message Authentication Code (MAC) must be calculated, using an SCM "Function Code 5000", and appended to the end of the message.

On receipt of a financial message from the remote node, the MAC must be verified, using an SCM "Function Code 5100", before any further processing can take place.

Acquirer Processing

The Interface process receives a STM/PSTM from the Authorisation process. The PIN block is encrypted under a session key.

Assuming that the remote node requires that the PIN block remains in encrypted form, the Interface process must translate the PIN block to encryption under the current outbound PIN key using an SCM "Function Code 6100" for ANSI PIN blocks or an SCM "Function Code 6180" for PIN/PAD PIN blocks and SCM "Function Code 6110" for AS2805.6.4 PIN Blocks.

The translated PIN block will then be inserted into an appropriate message to send to the remote node.

Issuer Processing

The Interface process receives a transaction request from the remote node. The PIN block is encrypted under the PIN Encryption Receive Key.

As BASE24 can handle the PIN block in encrypted form, it is not necessary to perform any translation of the PIN block.

The Interface process formats the following STM/PSTM fields for the appropriate Authorisation process:

PIN	->	Encrypted PIN Block in Ascii Hex char format
PIN-SIZE	->	"16"
PIN-FRMT	->	INTERFACE.PIN-BLK
PIN-KEY	->	current inbound pin key in binary format
PINPAD-CHAR	->	INTERFACE.PINPAD-CHAR
ANSI-OFST	->	INTERFACE.ANSI-PAN

Once the STM/PSTM is built, the Interface process sends the information to the Authorisation process for verification.

Section16

Authorisation Process

The Authorisation process determines whether a particular transaction is to be authorised at BASE24, at a host or at a switch.

HOST Authorisation

The Host Interface process receives the request from the Authorisation process, uses the SCM to translate the PIN block from encryption under the session key to encryption under the host session key and then sends the request to the host for verification.

Note: The ANSI and ISO Host interfaces currently do not support the SCM.

Switch Authorisation

The Switch Interface process receives the request from the Authorisation process, uses the SCM to translate the PIN block from encryption under the session key to encryption under the switch session key and then sends the request to the switch for verification.

BASE24 Authorisation

The SCM supports two types of PIN verification (i.e. DES (IBM 3624) PIN Verification and VISA PVV Verification).

The Authorisation process determines the type of PIN verification necessary by accessing PIN-VRFY-TYP of the appropriate IDF record. As the PIN is encrypted, the Authorisation process uses the SCM and data from the relevant KEYA record to verify the PIN.

If the PIN-VRFY-TYP is "01", then the Authorisation process invokes the Security Utility PIN^VRFY^IBM^DES. If the PIN Block is in ANSI format, an SCM "Function Code 6000" will be necessary to verify the PIN. If the PIN block is in PIN/PAD format, an SCM "Function Code 60A0" will be used. If the PIN block is in AS2805 6.4 format, an SCM "Function Code 6010" will be used.

If the PIN-VRFY-TYP is "04", then the Authorisation process invokes the Security Utility PIN^VERIFY^ABA^VISA. If the PIN Block is ANSI format, an SCM "Function Code 6001" will be necessary to verify the PIN. If the PIN block is in PIN/PAD format, an SCM "Function Code 60A1" will be used. If the PIN block is in AS2805 6.4 format, an SCM "Function Code 6010" will be used.

If the PIN-VRFY-TYP is not one of the above, an error message is logged describing the fault.

The Authorisation process also supports CVV/CVC verification if required by the institution. The security routine SCM^CARD^VERIFY is called to provide this support.

PIN change transactions are also supported by the ATM Authorisation process where the pin block is in PIN/PAD format and PIN verification method is VISA PVV. Function code "60B4" is called to calculate the new PVV.

Section 17

MDS Interface Process

The BASE24 MasterCard Debit Switch (MDS) ISO Interface has been upgraded to support the SCM module. It is important to note that the SCM module does not standardly support the MDS key management scheme and it needs to have extra functionality provided by the vendor.

SCM Functions for MDS Support

SCM Console

The SCM must provide a means of encrypting the KEK under an appropriate KM variant to be stored on the BASE24 database (i.e KEYF file).

The SCM has provided a console function in which KEK can be entered as a number of components and provided to the user encrypted under KM variant 40.

i.e. e*KMV40 (KEK) where variant 40 = x'0606060606060606'.

SCM Online Functions

The following function is the minimum which needs to be implemented by the SCM vendor to support the MDS environment:

Pin Key Translate		
ekek(kpe)	->	ekmv2 (kpe) for outbound PIN processing
		ekmv5 (kpe) for inbound PIN processing
		kcv (kpe) kvc for PIN encryption key

ERACOM have provided the above function together with extra functions for generating a PIN key and MAC key processing. Whilst all the following functions have been implemented, only the PIN translate is called by the MDS module.

D010 KPE-GEN
This function randomly generates a KPE encrypted under a single length KEK, encrypted under the appropriate KM variants and provides the KVC of the KPE.

Input:	e*KMV40(KEK)
Output:	eKEK(KPE) e*KMV2(KPE) e*KMV5(KPE) KVC(KPE)

D011 KPE-REC
This function translates a KPE encrypted under a single length KEK to encryption under the appropriate KM variants and provides the KVC of the KPE.

Input:	e*KMV40(KEK) eKEK(KPE)
Output:	e*KMV2(KPE) e*KMV5(KPE) KVC(KPE)

D012 KMAC-GEN
This function randomly generates a KMAC encrypted under a single length KEK, encryption under the appropriate KM variants and provides the KVC of the KMAC.

Input:	e*KMV40(KEK)
Output:	eKEK(KMAC) e*KMV1(KMAC) e*KMV4(KMAC) KVC(KMAC)

D013 KMAC-REC
This function translates a KMAC encrypted under a single length KEK to encryption under the appropriate KM variants and the KVC.

Input:	e*KMV40(KEK) eKEK(KMAC)
Output:	e*KMV1(KMAC) e*KMV4(KMAC) KVC(KMAC)

MDS Changes

The MDS module has been changed in two locations to support the SCM module: reading and writing the KEYF and performing the Key translate.

On initialisation and warmboot the MDS Interface process will test for the type of security module in place. If the SCM module is being used, it will retrieve the current PIN session keys and key exchange key from

KEYF.INTERFACE.E-KEKR
KEYF.INTERFACE.SCM.OUTBOUND.PIN.KEY1
KEYF.INTERFACE.SCM.OUTBOUND.PIN.KEY2
KEYF.INTERFACE.SCM.INBOUND.PIN.KEY1
KEYF.INTERFACE.SCM.INBOUND.PIN.KEY2.

On a key translation the MDS Interface process will test for the type of security module in place. If the SCM module is being used it will call "SCM^KEY^TRANSLATE" which will call a function to perform KPE-REC as defined above.

Note that the above session keys cannot be displayed through the KEYF AFT screens.

MDS Set UP

To set up the MDS Interface process the KEYF will need to be updated so that the 'eKEKr' contains the encrypted version of the KEK. This field is typically a double length key but in the MDS case it is only a single length. Therefore, only the two fields need to updated (i.e the KM ID portion and the left part of the key).

An example follows

```
BASE24-BASE  KEY FILE                ____  ____  __/__/__  __:__  01 OF 04
         DPC/FIID: ____         INTERFACE PROCESS: ________________

    ENCRYPT TYPE: _ (______________)    BASE24 ENCRYPT TYPE: _ (______________)
PIN BLOCK FORMAT: _ (____)                  ANSI PAN FORMAT: _ (________________)
MAC ENCRYPT TYPE: _ (______________)      PIN PAD CHARACTER: _
   MAC DATA TYPE: _ (_____)                  NUMBER OF KEYS: _ (_______)
      KEY LENGTH: _ (______)              FULL MESSAGE MAC: _ (________________)

                            INTERMEDIATE KEYS
          CLEAR: ________________                      CHECK DIGITS: ____
      ENCRYPTED: ________________

                              EXCHANGE KEYS
        PIN KEY: ________________  ________________   CHECK DIGITS: ____
        MAC KEY: ________________  ________________   CHECK DIGITS: ____
        KEKs: __ ________________  ________________   KVC: ______
        KEKr: 00 0123456789ABCDEF  ________________   KVC: ______

 RECORD LAST CHANGED: __/__/__  __:__   BY USER: ____ , ________   ADD
 *********************************** BASE24 ***********************************
  NEW PAGE:        FILE DESTINATION:        NEW LOGICAL NETWORK ID:
                         F12-HELP
```

Applied Communications, Inc.

Appendix A

Overview of SCM Keys on BASE24

SCM Keys Stored on File

	Key	Length	Description
Node:	e*KMV7(*KEKs)	17	Send Key Encrypting Key
	e*KMV8(*KEKr)	17	Receive Key Encrypting Key
	KPr	128	Receiver's Public Key
ATM:	e*KMVA(A)	9	A Key
	e*KMV5(M)	9	Master Key
	* e*KMV3(C)	9	Communications Key
	* e*KMV5(C)	9	Communications Key
	* e*KMV6(C)	9	Communications Key
POS:	e*KMV9(PPASN)	9	PIN Pad Security Number
	e*KMV7(*KEK1)	17	Key Encrypting Key 1
	e*KMV7(*KEK2)	17	Key Encrypting Key 2
	e*KMV1(KMACs)	9	MAC Send Key
	e*KMV3(KDs)	9	Data Send Key
	e*KMV4(KMACr)	9	MAC Receive Key
	e*KMV5(KPEr)	9	Pin Encryption Receive Key
	e*KMV6(KDr)	9	Data Receive Key
	# e*KMV6(*KMPP)	17	PIN Pad Master Key
	# e*KMV1(KMACH)	9	Housekeeping MAC Key
KEY:	# e*KMV8(*KI)	17	Acquirer Initialisation Key

Note: # - Currently not implemented.

SCM KEYS STORED IN MEMORY

	Key	Length	Description
Node:	e*KMV1(KMACs)	9	MAC Send Key
	e*KMV2(KPEs)	9	Pin Encryption Send Key
	e*KMV3(KDs)	9	Data Send Key
	e*KMV4(KMACr)	9	MAC Receive Key
	e*KMV5(KPEr)	9	Pin Encryption Receive Key
	e*KMV6(KDr)	9	Data Receive Key
	e*KEKr(KRr)	8	Random Key Inverted

Appendix B

SCM Function Calls

System Initialisation

C000 GETPUBLIC
This function returns a public key for the particular SCM

Input:	i
Output:	KPs

TERMINAL KEYS

ATM Terminal Initialisation

3600 ABKEYGEN
This function randomly generates a random ATM A-Key and B-Key. The A-Key is returned encrypted under *KMV10 and a *KTK for transmission to the terminal. The B-Key is returned encrypted under *KMV11 and a *KTK.

Input:	n		KTK index
Output:	e*KTK(A)	\	Send to terminal
	e*KTK(B)	/	
	e*KMV10(A)	\	Store in TDF
	e*KMV11(B)	/	

ATM Terminal Key Generation

3610 CKEYGEN
Note: Currently not implemented.

This function randomly generates a random ATM Communications Key and returns is encrypted under *KMV11, KMV6 for use when decrypting data, KMV3 when encrypting data and original C for transmission to the terminal.

Input:	e*KMV11(C)		Old Communications Key
Output	eC(C')		Send to terminal
	e*KMV11(C')	\	
	e*KMV3(C')	|	Store in TDF
	e*KMV6(C')	/	

3620 MKEYGEN
This function randomly generates a random ATM Master Key and returns is encrypted under KMV10 for use if Master Key needs to roll, KMV5 for use when sending financial transactions to AUTH and Master Key for transmission to the terminal.

Input:	e*KMV10(A)		ATM A-key
Output:	eA(M)		Send to terminal
	e*KMV10(M)	\	Store in TDF
	e*KMV5(M)	/	

POS Terminal Initialisation

3130 TERMKEYINIT (6.4)
This function randomly generates a set of initial keys and data for a terminal (*KEK1,*KEK2,PPASN). These are encrypted under *KTK for transmission to the terminal and the appropriate variants of KM for storage on file.

Input:	n		KTK index
			PPID
Output:	e*KTK(*KEK1)	\	
	e*KTK(*KEK2)	|	send to terminal
	e*KTK(PPASN)	|	
	e*KTK(PPID)	/	
	e*KMV7(*KEK1)	\	
	e*KMV7(*KEK2)	|	store on KSF
	e*KMV9(PPASN)	/	

POS Terminal Key Change

3000 TERMKEYGEN1
This function randomly generates terminal session keys, encrypting them under a Domain Master Key variant and the Terminal Master Key (*KEK1). The active Terminal Master Key (*KEK1) is updated before it is used to encrypt the terminal session keys.

Input:	e*KMV7(*KEK1		
	e*KMV9(PPASN)		
Output:	e*KEK1V1(KMACs)	\	
	e*KEK1V3(KDs)	|	
	e*KEK1V4(KMACr)	|	send to terminal
	e*KEK1V5(KPPr)	|	
	e*KEK1V6(KDr)	/	
	e*KMV1(KMACs)	\	
	e*KMV3(KDs)	|	
	e*KMV4(KMACr)	|	store on file
	e*KMV5(KPPr)	|	
	e*KMV6(KDr)	/	
	e*KMV7(*KEK1)		store on file

3010 TERMKEYGEN2
This function randomly generates terminal session keys, encrypting them under a Domain Master Key variant and the new Terminal Master Key (*KEK1). The active Terminal Master Key (*KEK1) is updated using the backup Terminal Master Key (*KEK2) prior to encrypting the session keys. The backup Terminal Master Key (*KEK2) is also updated during this procedure.

Input:	e*KMV7(*KEK2)		
	e*KMV9(PPASN)		
Output:	e*KEK1V1(KMACs)	\	
	e*KEK1V3(KDs)	|	
	e*KEK1V4(KMACr)	|	send to terminal
	e*KEK1V5(KPPr)	|	
	e*KEK1V6(KDr)	/	
	e*KMV1(KMACs)	\	
	e*KMV3(KDs)	|	
	e*KMV4(KMACr)	|	store on file
	e*KMV5(KPPr)	|	
	e*KMV6(KDr)	/	
	e*KMV7(*KEK1)		store on file
	e*KMV7(*KEK2)		store on file

POS Terminal Proof of End-Point

E000 TERMPROOF (6.4)
This function performs the cryptographic part of the terminal proof-of-endpoint processing. The PPASN is encrypted by the Key Encrypting Key (KEK) and the left hand 32 bits compared with the value received from the terminal.

Input:	e*KMV7(*KEK)	
	e*KEK(PPASN)	receive from terminal
	e*KMV9(PPASN)	
Output:		

E010 HOSTPROOF (6.4)
Note: Currently not implemented.

This function performs the cryptographic part of the host proof-of-endpoint processing. The left hand 32 bits of OWF(*KEK,PPASN) is generated and sent to the terminal for its verification.

Input:	e*KMV7(*KEK)
	e*KMV9(PPASN)
Output:	OWF(*KEK,PPASN)

NODE KEYS

Node Initialisation

Currently node initialisation is only supported through a manual exchange.

C100 NODEKEKGEN
This function randomly generates a Send Key Encrypting Key (*KEKs), encrypting it under a Domain Master Key variant and the Public Key.

Input:	KPr	
Output:	e*KMV7(*KEKs)	store on file
	cKPr(cKSs(*KEKs))	securely transport to i/c node
	KVC(*KEKs)	securely transport to i/c node

C110 NODEKEKTRANS
This function decrypts a Key Encrypting Key from a remote node and re-encrypts it under a Domain Master Key variant.

Input:	cKPr(cKSs(*KEKr))	securely receive from i/c node
	KVC(*KEKr)	securely receive from i/c node
Output:	e*KMV8(*KEKr)	store on file

Node Key Change

3200 NODEKEYGEN (6.3)

This function randomly generates session keys for use between nodes, encrypting them under a Domain Master Key variant and the Send Key Encrypting Key (*KEKs).

Input:	e*KMV7(*KEKs)		
Output:	e*KEKsV1(KMACs)	\	
	e*KEKsV2(KPEs)	|	send to i/c node
	e*KEKsV3(KDs)	/	
	e*KMV1(KMACs)	\	
	e*KMV2(KPEs)	|	store on file
	e*KMV3(KDs)	/	

4000 RTMK (KEY TRANSLATION - RECEIVE)

This function decrypts a set of session keys encrypted under a Receive Key Encrypting Key (*KEKr) and re-encrypts them under a Domain Master Key variant.

Input:	e*KMV8(*KEKr)		
	e*KEKrV1(KMACs)	\	
	e*KEKrV2(KPEs)	|	receive from i/c node
	e*KEKrV3(KDs)	/	
Output:	e*KMV4(KMACr)	\	
	e*KMV5(KPEr)	|	store on file
	e*KMV6(KDr)	/	

7000 KVCREQSIN (KEY VERIFICATION CODE)

This function returns the Key Verification Code for the single length keys provided.

Input:	i	variant required
	n	number of keys
	eKMVi(K)	encrypted key
Output:	KVC(K)	KVC of key

Node Proof of End-Point

E020 NODEPROOF

This function performs the request part of the internodal proof-of-endpoint processing. A single length random key is generated and forwarded to the remote node in normal and inverted mode encrypted by the Send Key Encrypting Key (KEKs).

Input:	e*KMV7(*KEKs)

Output:	e*KEKsV7(KRs)	send to i/c node
	e*KEKsV8(KRr)	

E030 NODERESP
This function performs the response part of the internodal proof-of-endpoint processing. The single length key (KRs) in the request is decrypted under the Receive Key Encrypting Key (KEKr) and re-encrypted in inverted mode. This result is compared to the value received from the remote node.

Input:	e*KMV8(*KEKr)	
	e*KEKrV7(KRs)	receive from i/c node
Output:	e*KEKrV8(KRr)	send to i/c node

MESSAGE AUTHENTICATION

5000 MACGEN (6.3 & 6.4)
This function calculates the MAC for a given message to be transmitted to another node (host or terminal).

Input:	e*KMV1(KMACs)
	n
	data
Output:	MAC(data)

5100 MACVERIFY (6.3 & 6.4)
This function verifies that the supplied MAC is correct for the data received.

Input:	e*KMV4(KMACr)
	MAC(data)
	n
	data

PIN MANAGEMENT

PIN Verification

6000 PINVERIFY (6.3)
This function verifies a PIN according to the DES (IBM3624) method. The PIN block shall be in ANSI format. The transaction originated at an interchange node.

Input:	maxpin	
	chklen	
	e*KMV5(KPEr)	
	eKPEr(PIN)	receive from i/c node
	n	
	PAN1	
	PAN2	
	offset	

6001 PINVERIFY VISA (6.3)
This function verifies a PIN according to the VISA PVV method. The PIN block shall be in ANSI format. The transaction originated at an interchange node.

Input:	e*KMV5(KPEr)	
	eKPEr(PIN)	receive from i/c node
	n	
	TSP12	
	PAN2	
	PVV	

6010 PINVERIFY (6.4)
This function verifies a PIN according to the DES (IBM3624) method. The PIN block shall be in ANSI format. The transaction originated at a terminal.

Input:	maxpin	
	chklen	
	e*KMV5(KPPr)	
	STAN	
	amount	
	eKPEr(PIN)	receive from terminal
	n	
	PAN1	
	PAN2	
	offset	

6011 PINVERIFY VISA (6.4)
This function verifies a PIN according to the VISA PVV method. The PIN block shall be in ANSI format. The transaction originated at a terminal.

Input:	e*KMV5(KPPr)	
	STAN	
	amount	
	eKPEr(PIN)	receive from terminal
	n	
	TSP12	
	PAN2	
	PVV	

60A0 PINVERIFY IBM3624 PIN BLOCK
This function verifies a PIN according to the DES (IBM3624) method. The PIN block shall be in PIN PAD format. The transaction originated at a terminal.

Input:	maxpin	
	chklen	
	e*KMV5(KPP)	
	eKPP(PIN)	receive from terminal
	n	
	PAN1	
	offset	

60A1 PINVERIFY VISA IBM3624 PIN BLOCK
This function verifies a PIN according to the VISA PVV method. The PIN block shall be in PIN PAD format. The transaction originated at a terminal.

Input:	e*KMV5(KPP)	
	eKPP(PIN)	receive from terminal
	n	
	TSP12	
	PVV	

PIN Translation

6100 PINBLOCKTRANS (6.3 to 6.3)
This function translates a PIN block from encryption under an PIN Encrypting Key (KPEr) to encryption under the Send PIN Encrypting Key (KPEs) of an interchange node.

Input:	e*KMV5(KPEr)	
	eKPEr(PIN)	
	PAN2	
	e*KMV2(KPEs)	
Output:	eKPEs(PIN)	send to i/c node

6180 PINBLOCKTRANS (IBM3624 to 6.3)
This function translates a PIN block from encryption under an ATM PIN Encrypting Key (KPEr) to encryption under the Send PIN Encrypting Key (KPEs) of an interchange node.

Input:	e*KMV5(KPEr)	
	eKPEr(PIN)	receive from terminal
	PAN2	
	e*KMV2(KPEs)	
Output:	eKPEs(PIN)	send to i/c node

6110 PINTRANS (6.4 to 6.3)

This function translates an ANSI PIN block from encryption under the Receive PIN Encrypting Key (KPEr) of the terminal to encryption under the Send PIN Encrypting Key (KPEs) of an interchange node.

Input:	e*KMV5(KPPr)	
	STAN	
	amount	
	eKPEr(PIN)	receive from terminal
	PAN2	
	e*KMV2(KPEs)	
Output:	eKPEs(PIN)	send to i/c node

PIN Change

PVV Generation

60B4 PVVGEN

This function generates a four digit PIN Verification Value for cards using the VISA method of PVV generation. The PIN block format shall be PIN/PAD.

Input:	e*KMV5(KPEr)
	eKPEr(PIN)
	n
	TSP12
Output:	PVV

CARD VERIFICATION

CVV Verification

8010 CVVVERIFY

This function verifies a CVV or CVC according to the VISA method.

Input:	e*KMV4(*KCVV)	CVV generation key from KEYA
	data	CVV data input
	n	number of CVV digits from KEYA
	CVV	

Applied Communications, Inc.

Appendix C

SCMCOM

Overview

This section describes the Security Control Module Command Interface or SCMCOM. The SCM is primarily designed to be used with software suites requiring hardware cryptographic protection. The operation of the SCM is then (mainly) hidden or restricted to certain software modules. However, it is a requirement to be able to check, using hardware and software methods, that the data being processed by the SCM is verifiably correct. To this end the SCMCOM utility is provided. The SCMCOM utility can generate results using the SCM or Software DES methods. The software DES methods rely upon library calls already implemented by ACI in supplied object libraries. SCMCOM can also be used to check and update the BASE24 KEYA and KEYF files. The KEYA file contains the KCVV key and the KEYF file contains the KEKs, KEKr and partner public key(which is not currently used by any application). These keys can be verified/updated using SCMCOM.

To run SCMCOM:

```
[ ASSIGN LCONF,  $VOLUME.SUBVOL.LCONF ]
[ RUN ] SCMCOM [ / IN <KMID-FILE> / ] [ <BLOCKER> ]
```

Where:

<KMID-FILE>	This is an optional EDIT file containing KM0 & KM1 in ASCII/HEX characters. This file contains two lines each lines each of 32 characters. (Line # 1 is KM0, #2 is KM1). The characters 0-9, A-F may be used. If the file is not present SCMCOM will use its own (default) KMIDs for software DES. This file can be selected after startup using the option from the Main Menu.
<BLOCKER>	This is an optional process name of a Blocker process already executing. If present, SCMCOM will attempt to open this process for subsequent communication. If this is not present SCMCOM will still startup but only certain functions will be available. This process can be selected after startup using the option from the Main Menu.

The option Guardian ASSIGN command can be used to specify the LCONF file to be used to locate the KEYA and KEYF files. The LCONF file can also be selected from the Main Menu.

Features

The SCMCOM utility is a menu driven interface to the SCM and software DES routines. Function call results from previous operations can be stored internally with a nominated variable name and then used for later actions. The output from SCMCOM can also be directed to a spooler output file and then printed for comparison purposes. All prompts, whether for Hex or Decimal data, indicate the maximum number of bytes that may be entered. In the case of Hex data this is usually 16 or 32 bytes. Leading zeroes are significant and must be entered. The operator may exit from most prompts, menu or data, by entering Ctrl-Y. Certain SCM functions return large amounts of data which can become inconvenient to write down or remember. SCMCOM allows the operator to store this data in a nominated file that may be used in later computations or transported to other sites. The KVCs of keys within the KEYA and KEYF files can be verified and updated with KVCs generated from the SCM if required.

Main Menu

The Main Menu appears as follows:

```
EXT Exit
SCM Security Control Module
CRM Software Encrypt/Decrypt
LDD Load Key File Data
CHG Specify Blocker Process
SPL Open Spooler File
LCF Specify and Open LCONF
HLP Display Help Information
ATM ATM Functions
POS POS Functions
```

The **SCM** option will cause the SCM menu to be displayed, although the functions from this menu will not be available if a Blocker process has not been nominated.

The **CRM** option will cause the CRM menu to be displayed.

The **LDD** option prompts the operator for an EDIT format file name containing two KMIDs in ASCII/Hex format to be used instead of the default internal ones. Refer to KEYFILE provided in BA51SCM subvolume.

The **CHG** option allows the operator to select a Blocker process to pass on requests to an SCM.

The **SPL** option directs SCMCOM to print results from functions to the spool file '$S.#SCMCOM'.

The **LCF** option allows the operator to specify the location of an LCONF configuration file which SCMCOM will use to find out where the KEYA and KEYF files are to be found.

If the operator types 'EXT', Return or Ctrl-Y then SCMCOM will terminate.

The **HLP** option displays the help facility.

The **ATM** option supports customer requirements for an ATM environment. Note if the customer does not have BASE24-atm this function will be disabled.

The **POS** option supports customer requirements for an POS environment. Note if the customer does not have the BASE24-pos AS2805 Device Handler module this function will be disabled.

SCM Menu

The SCM Menu appears as follows:

```
EXT Exit
VC1 KVC Single Key Generate
VC2 KVC Double Key Generate
KMT KM Translation
GPB Get Public Key
KSE Generate Kek Send Key
KSR Translate KeK Recv Key
EEE Echo Test
CKA Check/Update KVCs on KEYA
CKF Check/Update KVCs on KEYF
INP Install Public Key to KEYF
SKF Check Single KVC on KEYF
CKG Chess Kek Generate
CKR Chess Kek Receive
```

The **VC1** function returns the Key Verification code (KVC) of those single length keys encrypted by KM in the request.

The **VC2** function returns the Key Verification code (KVC) or those double-length keys encrypted by KM in the request.

The **KMT** function translates double-length keys from encryption under a previous Domain Master Key to encryption under the current KM.

The **GPB** function returns the Public Key/Modulus combination and optionally writes it to a file. The file is unstructured and the data is in ASCII display hex format for 256 bytes.

File Format:	Unstructured		
Record Layout:	01	PUBLIC-KEY	PIC X(256)

The **KSE** function generates a key for use as the Cross Domain Send Key and encrypts it under the Domain Master Key and doubly ciphers it under the sender's and receiver's RSA keys. This command requires as input the Public key of the receiver through a file, which has the same format as that produced by 'GPB'. This function will also write to a file the results of this SCM call. The file is unstructured and the data i.e the doubly ciphered Cross Domain Send Key and the KVC of the Key, is in ASCII display format for 138 bytes.

File Format:	Unstructured		
Record Layout:	01	cKPr-cKSs-KEKs	PIC X(132)
	01	KVC-KEKs	PIC X(6)

It will also prompt for a KEYF record to be updated with the Cross Domain Send key, if required.

The **KSR** function generates a key for use as a Cross Domain Receive Key from a remote node and encrypts it under KM for storage. The key will only be returned if the KVC is correct. This function expects as input the public key of the sender in a file of the same format as that of GPB and the doubly ciphered Cross Domain key and KVC in a file of the same format as that of KSE. It will also prompt for a KEYF record to be updated with the Cross Domain Receive key, if required.

The **EEE** option is an Echo Test to ensure all connections are valid.

The **CKA** option will read through the KEYA file, located by the specified LCONF file, and calculate the KVC for the KCVV Key which is encrypted under KMV4. If the KVC is null or blank it will be automatically updated on the file and the details displayed. If the KVC in the file does not match the one generated by the SCM or the KMID is invalid, the record details and correct and incorrect KVC will be displayed. If the SCM cannot generate the KVC because the record data is invalid, a message reflecting the SCM error will be displayed and no SCM KVC will be generated.

The **CKF** option will read the ICF and KEYF file, located by the specified LCONF file. This function will read an ICF record and try to read the corresponding KEYF record. It will determine the type of keys present by the interchange type specified in the ICF. If the KVC is null or blank it will be automatically updated on the file and the details displayed. If the KVC in the file does not match the one generated by the SCM or the KMID is invalid, the record details and the correct and incorrect KVCs will be displayed. If the SCM cannot generate the KVC because the record data is invalid, a message reflecting the SCM error will be displayed and no SCM KVC will be generated.

The **INP** option is designed to retrieve a public key from a file and install/update a specific KEYF record with that key. This key will usually be a interchange partner’s key. The user is prompted for the file name containing the Public Key. The KEYF record is retrieved by using the key details, FIID (Financial Institution ID) and the process name. If the record is retrieved successfully, then it is updated and re-written.

The **SKF** option is designed to check and update a single record from the KEYF file. The user is prompted firstly for the KEYF record key details, FIID and process. If the record is found then the user selects Key Length (double or single), and variants for KEKs and KEKr, these may default to KMV7 and KMV8 respectively. The KVCs are then generated and updated if blank or null or displayed if different.

The **CKG** option is designed to Generate KEKs and KEKr and return them encrypted under variants of KTK and KM.

The **CKR** option is designed to Generate KEKs and KEKr and return them encrypted under variants of KTK and KM.

Software DES Menu

The Software Encryption/Decryption menu appears as follows:

```
EXT Exit
ESK Encrypt 64 bit data value using Single Length Key
DSK Decrypt 64 bit data value using Single Length Key
EDK Encrypt 64 bit data value using Double Length Key
DDK Decrypt 64 bit data value using Double Length Key
XOR Exclusive OR
ONE One Way Function
STO Store Key
MAC Mac Generate
PAR Parity Correct
SHO Stored Keys
DEL Delete Stored Keys
KMV Key Variants
CPK Calc KPE for AS2805.6.4 Pin processing
KML Load KM Variants
CVV CVV/CVC Calculation
PIN IBM DES Pin Generation
DDD Decrypt 128 bit data value using Double Length Key
DDE Encrypt 128 bit data value using Double Length Key
```

The **ESK** function encrypts a 64 bit data value with a single length key.

The **DSK** function decrypts a 64 bit data value with a single length key.

The **EDK** function encrypts a 64 bit data value with a double length key.

The **DDK** function decrypts a 64 bit data value with a double length key.

The **XOR** function performs an exclusive OR on a supplied 64 bit data value using either a built in variant or user data.

The **STO** function allows the user to store user data in an internal table under a variable name for later use.

The **MAC** function creates a MAC value from a Clear key, Start Vector and Data.

The **PAR** function performs Parity correction on a 64 bit supplied value.

The **SHO** option displays the KMIDs, variants and user variables.

The **DEL** option deletes a user created variable from the internal table.

The **KMV** option creates new variants based upon those default values provided at startup.

The **CPK** option calculates a PIN Key used in formation of a AS2805 6.4 Pin Block.

The **KML** option loads Key Master variants.

The **CVV** option calculates CVV/CVCs.

The **PIN** option generates four character PINS.

The **DDD** option decrypts a double length value with a double length key.

The **DDE** option encrypts a double length value using a double length key.

ATM Functions Menu

The ATM Function menu appears as follows:

```
EXT Exit
ABK Generate AB Keys
```

The **ABK** function returns a random ATM A-key and B-key. Both keys are returned encrypted under a KTK key specified by user and under KMV10 and KMV11 respectively. The KTK format of the keys are optionally written to a file. The file is unstructured and the data is in ASCII display hex format for 32 bytes.

File Format:	Unstructured		
Record Layout:	01	e-KTK-A	PIC X(16)
	01	e-KTK-B	PIC X(16)

POS Functions Menu

The POS Function menu appears as follows:

```
EXT Exit
TKI Generate initial AS2805.6.4 Term Keys
```

The **TKI** function returns sets of initial keys and data for an AS2805 6.4 terminal *KEK1, *KEK2 and PPASN. These keys are returned encrypted under a KTK key specified by user and under KMV7 and KMV9 respectively. The KTK format of the keys are written to the DKSF file specified in LCONF. The encrypted versions of the keys are written to the KSF file specified in the LCONF.

Please refer to the AS2805 documentation for management of AS2805 6.4 terminal population and the use of this function.

Sample Operation

The following is an example of SCMCOM operation.

This example depicts the SCMCOM startup and external ASSIGNs, the Blocker process name is a command line option. The main menu appears at startup.

ASSIGN LCONF,$DATA6.PC.L1CONF << External LCONF Assign >>
RUN SCMCOM/IN KEYFILE/ $BKR1 << Run command inc. KM file & Blocker>>

```
SCMCOM Main Menu

EXT Exit
SCM Security Control Module
CRM Software Encrypt/Decrypt
LDD Load Key File Data
CHG Specify Blocker Process
SPL Open Spooler File
LCF Specify and Open LCONF
HLP Display Help Information
ATM ATM Functions
POS POS Functions
Command>SCM
```

The operator has selected the SCM menu, and this is now displayed.

```
Security Control Module Menu

EXT Exit
VC1 KVC Single Key Generate
VC2 KVC Double Key Generate
KMT KM Translation
GPB Get Public Key
KSE Generate Kek Send Key
KSR Translate KeK Recv Key
EEE Echo Test
CKA Check/Update KVCs on KEYA
CKF Check/Update KVCs on KEYF
INP Install Public Key to KEYF
SKF Check Single KVC on KEYF
Command>VC1
```

The operator has selected the VC1 option. This option will generate a single Key Verification code based upon a Variant, KMID and Key provided.

```
Single Key KVC Generation


Variant Required : 01

KMID Required    : 01

Encrypted Key    : 0123456789ABCDEF

KVC Generated for key     : CC1EA1            <---- Resulting KVC

Continue (Y/N)? N                             <---- User exits option

<< SCM Menu displayed >>

Command>GPB                                   <---- User selects GPB option
```

The operator selects the Get Public Key option.

```
Get Public Key
Public Key:
87598657F90BA6ECA5B70E65D7CA7C62D2294B04F39BD3EB0AA77ACD2D3751DE4D89954AE01280F138
41678C1329783939D21F767663007BB3E00E4D1B7ED731
0000000000000000000000000000000000000000000000000000000000000000000000000000000000
00000000000000000000000000000000000000008BAF

Write Data to File (Y/N)? Y                     <---- Users wants to save Key
Filename: PUBKEY                        <---- File Name selected

Data Written.  Return to continue:              << carriage return >>

<< SCM Menu displayed >>

Command>KSE                                     <---- User selects KSE option
```

The Generate KEK Send key option generates a cross domain send key for another interchange.

```
Encrypt Cross Domain Send Key

Public Key Required -:

Key Filename: PUBKEY                            <---- Filename with key to
                                                <---- to be read in.

Cross Domain Send Key       : 00C4309353A046CEBCEBD56471C83147C6

Dbl Cipher Cross Domain Key :
3A34481BFDDA9AE78D7D123ABCE57F4CD66032479BF96342AFB3C3CF91D01732F5
6EE7273606C4BC2C0631D2427FB6769252943318185C3262E8E8709734DAEB94DF

Verification Code            : 1D7EB0

Write Data to File (Y/N)? Y                     <---- User wants to save data
Filename: XKEY                                  <---- File Name for key

File does not exist, Create (Y/N)? Y            <---- File must be created

Data Written.  Return to continue:              <<< Carriage Return >>>

<< SCM Menu displayed >>
```

User selects Check KEYA file KMV4, record type 11 records.

```
Command>CKA

KVC Check Digit for KEYA


SCM detected error, text is:                    <---- The SCM did not like
----------------------------------------        <---- Selected KM
KM Requested Not Held
----------------------------------------

KVCs mis-match or SCM error, data follows:

Record Key:

       Record Type : 11
       Group       : FI01
       Begin Date  : WWWWWW
       End   Date  : WWWWWW

KMV4 KMID :20
KMV4 Key  :20202020202020202020202020202020
KMV4 KVC  :202020

Type Return to continue -               <<< Carriage Return >>>

KEYA Check digit update completed


008  KEYA Records Read                          <---- Totals
000  KEYA Records Updated


Type Return to continue -
```

```
<< SCM Menu displayed >>
```

The user selects the CKF option to check KMV7 & KMV8.

```
Command>CKF

KVC Check Digit for KEYF

KVCs mis-match or SCM error, data follows:

Record Key:

       Record Type : 1
       FIID        : 0001
       Product     : P1A^HISF

  KMV7 KMID :00
  KMV7 Key  :0123456789ABCDEF
  KMV7 KVC  :80CC95
* SCM  KVC  :A2EC8A

Type Return to continue -

KVCs mis-match or SCM error, data follows:

Record Key:

       Record Type : 1
       FIID        : 0001
       Product     : P1A^HISF

  KMV8 KMID :00
  KMV8 Key  :0000000000000000
  KMV8 KVC  :003030
* SCM  KVC  :FC1C5F

Type Return to continue -

KVCs mis-match or SCM error, data follows:

Record Key:

       Record Type : 1
       FIID        : 0001
       Product     : P1A^HISO

  KMV7 KMID :00
  KMV7 Key  :0000000000100000
  KMV7 KVC  :000100
* SCM  KVC  :5D4B12

Type Return to continue -

KVCs mis-match or SCM error, data follows:

Record Key:

       Record Type : 1
       FIID        : 0001
       Product     : P1A^HISO

  KMV8 KMID :00
  KMV8 Key  :1111111111111111
  KMV8 KVC  :3175C7
* SCM  KVC  :CBE52E

Type Return to continue -

>>> KVC Generated, data follows :<---- KVC generated for blank
                                                 <---- or null KVC

Record Key:

       Record Type : 1
       FIID        : 0002
       Product     : SIM^HISF

  KMV7 KMID :00
  KMV7 Key  :0000000000000000
  KMV7 KVC  :A44331
```

```
Type Return to continue -

KVCs mis-match or SCM error, data follows:

Record Key:

       Record Type : 1
       FIID        : 0002
       Product     : SIM^HISF

  KMV8 KMID :01
  KMV8 Key  :0000000000000000
  KMV8 KVC  :111111
* SCM  KVC  :AA46C5

Type Return to continue -

KVCs mis-match or SCM error, data follows:

Record Key:

       Record Type : 1
       FIID        : 0002
       Product     : SIM^HISO

  KMV7 KMID :00
  KMV7 Key  :000000000000DD00
  KMV7 KVC  :EC079F
* SCM  KVC  :E7012F

Type Return to continue -


<< etc >>

011  KEYF Records Read                          <---- Totals
001  KEYF Records Updated


Type Return to continue -               <<< Carriage Return >>>

<< SCM Menu displayed >>
```

Users selects Install Public Key Option.

```
Command>INP

Install Partner Public Key

Public Key Required -:


Key Filename: PUBKEY                         <---- Public Key File name

Public Key extracted from file.

Enter KEYF key details :

FIID        : 0001                      <---- Financial Institution
Process     : P1A^HISF                          <---- BASE24 Process

KEYF record read OK.

KEYF file updated OK.

Type Return to continue -               <<< Carriage Return >>>

<< SCM Menu displayed >>
```

Users updates single KEYF record.

```
Command>SKF

Check/Update KVC for single KEYF record

Enter KEYF key details:

FIID        : 0001                      <---- Financial Institution
Process     : P1A^HISO                          <---- BASE24 Process
```

```
KEYF record read OK.

(S)ingle or (D)ouble Keys: D                   <---- Use 32 bit keys

Variant Required for KeKs (default 7):         << carriage return >>
Variant Required for KeKr (default 8):         << carriage return >>

KVCs mis-match or SCM error, data follows:

Record Key:

       Record Type : 1
       FIID        : 0001
       Product     : P1A^HISO

  KMV7 KMID :00
  KMV7 Key  :00000000001000000000000100000000
  KMV7 KVC  :000100
* SCM  KVC  :FF4A8D

Type Return to continue -

Security Control Module Menu

EXT Exit
VC1 KVC Single Key Generate
VC2 KVC Double Key Generate
KMT KM Translation
GPB Get Public Key
KSE Generate Kek Send Key
KSR Translate KeK Recv Key
EEE Echo Test
CKA Check/Update KVCs on KEYA
CKF Check/Update KVCs on KEYF
INP Install Public Key to KEYF
SKF Check Single KVC on KEYF
Command>  << Carriage Return >>

SCMCOM Main Menu

EXT Exit
SCM Security Control Module
CRM Software Encrypt/Decrypt
LDD Load Key File Data
CHG Specify Blocker Process
SPL Open Spooler File
LCF Specify and Open LCONF
HLP Display Help Information
ATM Functions
Command>CRM

Software Encrypt/Decrypt Menu

EXT Exit
ESK Encrypt 64 bit data value using Single Length Key
DSK Decrypt 64 bit data value using Single Length Key
EDK Encrypt 64 bit data value using Double Length Key
DDK Decrypt 64 bit data value using Double Length Key
XOR Exclusive OR
ONE One way function
STO Store key
MAC Mac generate
PAR Parity correct
SHO Show stored keys
DEL Delete stored key
KMV Key variants
CPK Calc kpe for AS2805.6.4 Pin processing
KML Load km variants
CVV CVV/CVC Calculation
PIN IBM DES Pin generation
DDD Decrypt 128 bit data value using Double Length Key
DDE Encrypt 128 bit data value using Double Length Key
Command>ESK
```

User selects software encryption of 64 bit data value with single length key

```
Single Key Encryption

Data    : 0123456789ABCDEF               <---- 64 bit data

Key     : 0000111100001111               <---- 64 bit key
```

```
Result : 69A4 22ED 7011 7942                    <---- Result

Store Data (Y/N) ?Y                             <---- Store data internally

Key/Data Name (15 Chars Max): VAR1              <---- internal Variable name

Data Stored.


Continue (Y/N)? N
```

User selects software decryption of 64 bit data value with double length key

```
Command>DDK

Double Key Decryption

KMV Index : 01                                  <---- Key Variant to use
Data      : 0123456789ABCDEF                    <---- 16 bit data

Result: F342 5C01 C9E7 1E85

Store Data (Y/N)? Y

Key/Data Name (15 Chars Max): VAR2

Data Stored.


Continue (Y/N)? N
```

Generate Message Authentication Code

```
Command>MAC

Generate MAC Keys

Clear MAC Key              : 0010010100010000

Start Vector (Default 0's) : 1010101000000000

Data                       : ABAB3B34BB010000

Result : 074D 90EE C0B1 4825

Store Data (Y/N) ?N

Continue (Y/N) ?N
```

User displays Software and User Keys

```
Command>SHO

Show Stored Keys

Name                Key
KM Indicator        00
KM                  0011 2233 4455 66778899 AABB CCDD EEFF
V1                  2424 2424 2424 2424
V2                  2828 2828 2828 2828
V3                  2222 2222 2222 2222
V4                  4848 4848 4848 4848
V5                  4242 4242 4242 4242
V6                  4444 4444 4444 4444
V7                  8282 8282 8282 8282
V8                  8484 8484 8484 8484
V9                  8888 8888 8888 8888
V10                 AAAA AAAA AAAA AAAA
V11                 A6A6 A6A6 A6A6 A6A6
VAR1                69A4 22ED 7011 7942
VAR2                F342 5C01 C9E7 1E85

Continue (Y/N)? Y

KMV1                2435 0617 6071 4253 ACBD 8E9F E8F9 CADB
```

```
KMV2                    2839 0A1B 6C7D 4E5F A0B1 8293 E4F5 C6D7
KMV3                    2233 0011 6677 4455 AABB 8899 EEFF CCDD
KMV4                    4859 6A7B 0C1D 2E3F C0D1 E2F3 8495 A6B7
KMV5                    4253 6071 0617 2435 CADB E8F9 8E9F ACBD
KMV6                    4455 6677 0011 2233 CCDD EEFF 8899 AABB
KMV7                    8293 A0B1 C6D7 E4F5 0A1B 2839 4E5F 6C7D
KMV8                    8495 A6B7 C0D1 E2F3 0C1D 2E3F 4859 6A7B
KMV9                    8899 AABB CCDD EEFF 0011 2233 4455 6677
KMV10                   AABB 8899 EEFF CCDD 2233 0011 6677 4455
KMV11                   A6B7 8495 E2F3 C0D1 2E3F 0C1D 6A7B 4859

Type Return to continue - << carriage return >>
```

User exits SCMCOM

```
Command> << carriage return >>

SCMCOM Main Menu

EXT Exit
SCM Security Control Module
CRM Software Encrypt/Decrypt
LDD Load Key File Data
CHG Specify Blocker Process
SPL Open Spooler File
LCF Specify and Open LCONF
HLP Display Help Information
ATM Functions
POS Functions
Command> << carriage return >>

SCMCOM Terminated.
```

Applied Communications, Inc.

Appendix D

Configuring SCM Devices with Ethernet

Required Hardware/Software

Tandem Hardware/Software

Minimum requirements will be as follows:

- Tandem NonStop System (Guardian 90, SCF, SCP, MLMAN, MLAM).
- TLAM Controller - Ethernet or Token Bus MLMUX)
- **Ethernet** or Token Bus Link (Ethernet link includes transceiver)
- Local Area Network (LAN O/S and NETBIOS)
- TAL
- Ethernet Transceivers and Cabling

SCM Hardware

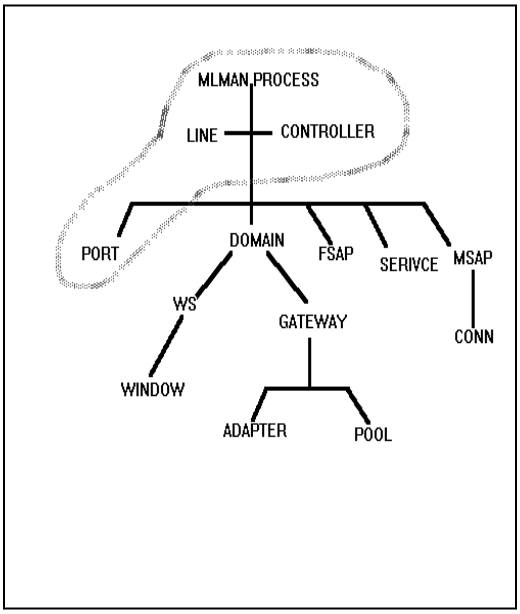
The ERACOM Security Control Module (SCM) is a physically secure device incorporating a set of logically secure cryptographic operations and some internal storage for any secret information required by these operations. The SCM should come with the following hardware items:

- The SCM unit including a power cord.
- The SCM programmers guide, SCM Console users guide, Maintenance & Installation Guide, Custom Manuals (if any) and Host Connection Supplements.
- Four sets of keys for the Power, Erase Memory and Battery compartment access, Enable Console and Chassis access.
- Cabling for the Wyse 50 terminal, Eracom null modems and adaptors for performing memory transfers between other SCMs.

Hardware/Port Configuration

SCF - TLAM/Ethernet

The MultiLan/Ethernet Card(s) will be installed by the Tandem Systems Engineer. The cabling, of the Tandem into the LAN & the SCM into the LAN will also be done by an Engineer. **SCF** is the Tandem Subsystem Control Facility. This product is used for maintaining various communications modules (not just Ethernet & Multilan). Although the MultiLan product is used, in reality only a small portion of this subsystem is required to be configured for Ethernet access. The following diagram (taken from the SCF help facility) indicates what needs to be configured/added:



The Ethernet ports are only a small part of the Multilan system and can be added quite independently as per the following example:

```
ASSUME LINE $LANA
ADD PORT #REC01, TYPE LLC1,
ADDRESS %H0004, DELIVERYMODE
SERIAL, REQUESTSIZE 31000,
DATAFORWARDTIME 0.060,
DATAFORWARDCOUNT 1
```

The above SCF commands will add a single Ethernet port which can be used by applications to pass data to the SCM device. The port is accessed like any other Guardian device. That is, standard system calls are used (READs WRITES etc.). In the above example the LINE or CONTROLLER is $LANA and the Ethernet port name is #REC01. The TYPE **must** be LLC1. This indicates that we are going to use Logical Link Control Type 1, which is IEEE 802.2 compliant and can be used with the SCM. In fact, the SCM can use either IEEE 802.2 or IEEE 802.3 (default). The IEEE 802.2 interface must be selected for use on the SCM console otherwise the Tandem will not communicate. More information concerning port options can be found Tandem Multilan/TLAM programming & reference manuals. The Dataforwardtime and Dataforwardcount control the delivery of inbound messages to a Tandem application.

SCM - Security Module

The SCM must be configured with its own Ethernet address and (optionally) the Ethernet addresses of the Hosts allowed to communicate with it. In fact, before the SCM will respond (in a meaningful way) with data, its functions must be selected to be available from the console Administration menu.

SCM Network Address & Host

The SCM should come from the factory with a unique Ethernet address already set up for the end user. This will be a 12 byte hexadecimal address. This address can be found on the **Configure Host Communications** option via the **SCM Administration Menu**. This screen presents two fields for the Local (SCM) address and up to ten fields for Host addresses. <u>The Host addresses may be left blank</u>, in which case any Host may talk to this particular SCM. The SCM local address should not really be changed, unless there is an address conflict on the LAN. However, the local address can be ensured to become the default address (by pushing F5), otherwise the SCM will not receive its messages.

Function Enabling

The SCM cannot be used (or be useful for that matter) unless its functions have been enabled from the SCM console. This can be achieved via the **SCM Administration Menu** then the **Enable Functions Menu**. This menu will present another list of sub functions each with their own screen. These screens are all quite similar. Each screen presents a list of 'check box' items to tick. Each 'check box' represents a specific SCM function which may (or may not) be enabled. The slash "/" character is used to toggle the selection. Note that these screens (as are most in the SCM) should be exited using the Enter or Return Key, especially if changes have been effected. Exitting by the F3 or F12 keys causes the changes to be lost.

Tandem Software

Tandem Supplied System Software

When the Multilan/TLAM software is purchased certain source and object files will be provided on a nominal system subvolume (usually) called **ZMLAM**, although this is up to the system administrator. There will also be an SCP type object file which is required to be started (running) to use SCF to Add or Alter any of the configurable details. This object is called OMLAM and must be started with a process name of **$ZLMG**. The Multilan/TLAM system itself will be Sysgened or COUPed in by the system administrator. For further information please consult the Multilan Management & Operations manual.

Blocker Driver & Applications

The Multilan Programming Manual provides programming examples and complete programs to perform communications at an applications level. This consists of coding an application that utilises the formatting functions provided. This application is known as a 'Blocker' process. A Blocker process has been developed for BASE24 and is required to give BASE24 applications access to the Eracom device without having to know the intricacies of Ethernet/LLC1. The application just Opens the Blocker process and writes/reads to it.
The Blocker process is started in one of two ways: :
RUN BLOCKERO/NAME $BLKR,NOWAIT, IN <CONFIG> ..../
or use the GOBLKRT Tacl file

The Blocker process just runs as a background job fielding requests from the BASE24 satellite programs.

LCONF Changes

The 'Blocker' program requires several parameters to be configured so that it can communicate correctly with the Ethernet line. These parameters include the physical line name(s) and Ethernet addresses of the SCM. Refer to sections 4 and 5.

Installation

Hardware

SCM

The SCM unit is rack mounted and requires a single power cable. Ports at the rear of the device may be connected into a patch panel so that switching can be performed. The SCM cannot be opened or tampered with or it will erase its memory and become useless. A high speed Ethernet cable will be required for the SCM to be accessed via the MultiLan system. The SCM will also require a WYSE 50 terminal to be connected (or similar) for access to its Administration system.

Tandem Multilan/Ethernet

The Tandem MultiLan system requires a card with the controller (product numbers 5600/3611) to be installed (plugged in) to the rack within the system being used. From the rear of this card a high speed serial cable will be attached to a junction box (T - Junction) on the LAN system. The Sysgen and subsequent configuration of the Ethernet lines are site dependant, see Tandem TLAM Configuration and Management Manual.

Software

SCM

The SCM will come pre-loaded with its software. The SCM Administration system is accessed using the attached WYSE 50 terminal. The terminal should initially display a Logon style screen requesting a password. The default password is **PASSWORD**, which can and should be altered. The initial menu displays a list of sub-menus etc. For further information consult the SCM Installation and Maintenance guide.

Tandem

The Tandem software (Multilan/TLAM) usually comes on magtape or cartridge and should be installed by a Tandem engineer or the systems administrator. If the Multilan/TLAM already exists then only the additional ports need to be added.

BASE24

The BASE24 'Blocker' software is provided as part of the standard SCM support by ACI. Multiple copies of the 'Blocker' object may be executing (under different process names) at the same time. They may all share the same LCONF file although parameters for each process are required.

Trouble Shooting

When the Software and Hardware is installed it is still quite likely communications to the SCM will fail because some (important) step has been 'forgotten'. The following is a list of some of the problems that may be encountered:

- Problem: **'NO SCM COMMUNICATION POSSIBLE'**
- Reasons: i) No SCM functions enabled.
  ii) ISO 802.2 type comms must be selected the **Configure Host Communications** menu, selected with a '/'.
  iii) Incorrect Host address inserted.
  iv) SCM Local address does not match address in LCONF file.
  v) Host address selected, but does not match actual Host address - check using SCF for LAN address.
  vi) Blocker process not running.
  vii) Blocker Ethernet Ports are either not configured or not started, - check SCF.
  viii) Blocker LCONF ports, addresses are wrong.

Applied Communications, Inc.

Appendix E

BASE24 BLOCKER Program

Parameters and Assigns

The following is a list of LCONF parameters and assigns that must be added for each 'Blocker' process. Please refer to the Tandem file BA51SCM.BKLRCONF. The Blocker process forms a BASE24 symbolic process name by using the PPD name minus the "$" (eg $BKR1 becomes BKR1).

NAME	VALUE	COMMENTS
BLKR-MAX-AGGR-WAIT-TIME	nn	Max. time in 0.01 secs to wait before dispatching an incomplete aggregate SDU packet. The time starts from the receipt of the first SDU request that builds the empty aggregate SDU. Default 0.02 secs.
BLKR-SCM-FOREIGN-ADDRESS-1	16 Hex Bytes	LAN address of security device.
BLKR-SCM-LINK-DEPTH	nn	The number of outstanding reads posted to the LAN, Max. 15, default 15. Ensure LAN DATA-FORWARD-COUNT param is set to 3->6 SDUS/packet.
BLKR-SDU-BLOCKING-COUNT	nn	Maximum number of requests (SDUs) that can be packed for outbound dispatch. Default 6.
BLKR-SEC-REPORT-NAME	Error report file	The name of the default logging device for error output, usually $0 or sudoterm.
BLKR-OPEN-RETRY-TIME	nn	Max time, in 0.01 secs to wait before attempting to re-open a closed port.
BLKR-RETRY-REPORT-FREQ	nn	How often Blocker issues port error message. i.e. '10' means every 10th time.
BLKR-TLAM-PORT-NAME	Tandem device or line name	The Tandem Port name accessing the Ethernet Lan.

Overview

The Blocker process provides an interface between all satellite processes requiring the SCM and the Ethernet LAN attached SCM devices. The Blocker process performs only as a protocol interface and does not examine the contents of the message except for the message sequence number in bytes 2:3 of the message. The Blocker process assembles requests into an aggregate service data unit (SDU) for greater communications efficiency. The LAN transports the aggregate SDU to the SCM, feeds the messages into the SCM and re-blocks the reply into the aggregate SDU response.

Technical Specifics

This section attempts to explain, in-depth, the workings and relationship between the (BASE24) satellite processes, SECUTILS security module, the Blocker process and the SCM itself:

```
+--------------------------------------+
|  Satellite Processes/ SECUTILS |
+--------------------------------------+
|                |                |
\                |               /
  \              |              /
        +--------------------+
        |      Blocker      |
        +--------------------+
          |              |
          +---------------+
          |  S C M(s)     |
          +---------------+
```

Starting from a satellite process a request buffer is constructed using (bound-in) calls from the SECUTILS module. This buffer will contain the SCM request function, in the format defined by the SCM reference manual and an <u>internal sequence number</u> generated by the SECUTILS routines. This sequence number is placed in the Message Id field (bytes 2:3) of the SCM function structure. This request is sent to the Blocker process. The Blocker process receives the request and stores this sequence number (from Secutils) in its internal receive table which is indexed by the message tag. This message tag is the lowest number available (starting from zero) and up to the receive-depth + 1 (currently 200). The message tag count decreases every time an outstanding request is replied to. This message tag replaces the sequence number (Message Id) in the SCM request message in bytes 2:3. The request message is written to the SCM. When a reply comes from the SCM the Message Id (which must be intact), is used as the index into the Blocker’s receive table to extract the original sequence number from the Satellite/SECUTILS. The Satellite is then replied to with the original sequence number, which it checks to ensure that it received the correct reply.

The Blocker process can 'block' together multiple Service Data Units (SDUs) as they are received and when the aggregate (combined) SDU is full (5 SDUs presently - LCONF param BLKR-SDU-BLOCKING-COUNT), it is sent to the SCM. A timer (LCONF param BLKR-MAX-AGGR-WAIT-TIME) is initiated on receipt of the first SDU and if the Aggregate SDU is not full and a timeout occurs the incomplete aggregate SDU is sent anyway. This ensures a fairly regular data flow to and from the Blocker and the SCM(s).

A particular Blocker process can communicate with many SCMs and one Blocker can have many outstanding requests on the one Ethernet port. This is dictated by LCONF parameters BLKR-SCM-FOREIGN-ADDRESS-n and BLKR-SCM-LINK-DEPTH. Each SCM has a unique Ethernet address on the LAN. When the Blocker process is deciding which SCM to send to, it determines which SCM has the least number of outstanding messages that have been sent to it, but not replied to, and sends (writes) the aggregate SDU to that SCM.

The Blocker opens the Ethernet port twice, for reading and writing. The number of outstanding write requests is (nominally) set to five, since you can have up to five SCMs configured to one Blocker. The number of read requests is set to five plus the LCONF link depth parameter. In other words, the Blocker process will always have outstanding reads on the Ethernet port to capture incoming data.

Applied Communications, Inc.

Appendix F

SECUTILS Log Messages

SECUTILS Log Messages Manual

Number: The number here refers to the EMS event number

Severity: The severity code indicates the seriousness of the log message. Error and Critical messages are shown in reverse-video when using the EMSPERUS utility to make them stand out from the less serious Warning and Informational messages. When a Critical message is logged, the logging process will typically abend.

Severity Codes:

0 = Informational

1 = Warning

2 = Error

3 = Critical

Text: The text here is the text as logged.

<error> is a Guardian error number.

<file> is the name of a security device, blocker process or boxcar process depending on the security device type.

Number: 9100

Severity: 1

Text:
```
Security device <file>, hardware error <error>
<file> = Security device name
<error> = Error number
```

Cause: Error has occurred while attempting to communicate with device

Remedy: Depends on <error>

Number: 9101

Severity: 3

Text:
```
Attempted access to a pin security device that is not
configured
```

Cause: Security device type does not exist.

Remedy: Correct security device type.

Number: 9102

Severity: 3

Text: `<Close on <file> failed - <error>`

`<file> = File name`

`<error> = Error number.`

Cause: An error occurred while the process was trying to close the identified file.

Remedy: Contact the system manager.

Number: 9103

Severity: 3

Text: `Open on <file> failed - <error>`

`<file> = File name`

`<error> = Error number.`

Cause: An error occurred while the process was trying to open the identified file.

Remedy: Contact the system manager.

Number: 9104

Severity: 3

Text: `<text> <error>`

`<text>   = Logical Network Configuration File (LCONF) error message`

`<error> = Error number`

Cause: An error occurred while reading Logical Network Configuration File (LCONF) and error return from lconf^read greater than 0.

Remedy: Contact the system manager.

Number: 9105

Severity: 3

Text: `<text>`

`<text> = Logical Network Configuration File (LCONF) error message`

Cause: An error occurred while reading Logical Network Configuration File (LCONF) and error return from lconf^read less than 0.

Remedy: Depend on error message.

Number:	9106
Severity:	1
Text:	`Switching to secondary security device <file>` `<file> = Security device name`
Cause:	Error has occurred when the first security device had fail and process had switch to secondary device .
Remedy:	Collect all information and contact HELP24.

Number:	9107
Severity:	1
Text:	`Invalid device subtype - defaulting to spaces`
Cause:	The device subtype as specified in the LCONF was invalid. The default subtype is spaces..
Remedy:	Correct the device subtype.

Number:	9108
Severity:	1
Text:	`Invalid <secure-dev-fail-notify> parameter - defaulting to 20`
Cause:	The fail notify of security device was invalid . The default limit is 20.
Remedy:	Correct the param in the Logical Network Configuration File (LCONF).

Number:	9109
Severity:	3

Text:	`Security device <file> message buffer content = <text>`
	`<file> = Security device name`
	`<text> = Visa security device's response message`

Cause:	The response message received from security device was invalid.
Remedy:	Check that the security device is in a valid state. If the problem still exists, collect all information and contact HELP24.

Number:	9111
Severity:	1
Text:	`Sanity Check/PIN Block Error - response = <text>` `<text> = device's response message`
Cause:	The Sanity Check error occurred at the security device.
Remedy:	Check that the security device is in a valid state. If the problem still exists, collect all information and contact HELP24.

Number: 9112
Severity: 3
Text: `Invalid parameter - Proc: <text>`
`<text> = secutils procedure name`
Cause: Invalid parameter.
Remedy: Correct parameter or contact HELP24.

Number: 9113
Severity: 3
Text: `Unknown security device type <text> - Proc: <text>`
`<text> =  Security device type/`
`<text> = secutils procedure name`
Cause: The device type was invalid in Logical Network Configuration File (LCONF).
Remedy: Correct the device type in Logical Network Configuration File (LCONF).

Number: 9114
Severity: 3
Text: `Invalid request for security device type <num> in proc: <text>`
`<text> = Secutils procedure name`
`<num> = Security device type.`
Cause: Invalid request for the specified security device.
Remedy: Verify that request is valid for the specified device.

Number: 9115
Severity: 3
Text: `Using prior configuration environment due to OPEN FAILURE(S)`
Cause: Open on the new security devices failed ,setting back to old security device.
Remedy: Verify that new security device is in a valid state.

Number: 9116
Severity: 3
Text: `You can't get here from there - proc: <text>`
`<text> = secutils procedure name`
Cause: Programatic error.
Remedy: Contact HELP23.

Number: 9117

Severity: 3

Text: `Failure limit exceeded on security device <file> WARMBOOT required`

Cause: The security device failed response to process and failure retry limit exceeded..

Remedy: Make sure that the security device is in a valid state and WARMBOOT process.

Number: 9118

Severity: 1

Text: `Invalid <secure-dev-tim-lim> parameter ...  defaulting to 5 seconds`

Cause: The timer limit of security device was invalid . The default limit is 5 seconds.

Remedy: Correct the param in the Logical Network Configuration File (LCONF).

Number: 9119

Severity: 3

Text: `Invalid security device assign filename <file> in proc: <text>`

`<file> = Security device file name`

`<text> = Secutils procedure name`

Cause: The security device assign file name was invalid.

Remedy: Correct the security device assign file name.

Number: 9120

Severity: 3

Text: `<file> error <error> <text>`

`<file> = Security device file name`

`<error> = An error number`

`<text> = The description of error`

Cause: An error occurred while the process check for critical error response return from security device.

Remedy: Depend on error.

Number: 9121

Severity: 3

Text: `Missing a required parameter in Proc: <text>`

`<text> = secutils procedure name`

Cause: The required parameter does not exists.

Remedy: Verify that all required parameter exists.

Number: 9122
Severity: 3
Text: `Invalid parameter in Proc <text>`
`<text> = secutils procedure name`
Cause: Invalid parameter.
Remedy: Correct parameter.

Number: 9123
Severity: 3
Text: `Missing ASM parameter in Proc: <text>`
`<text> = secutils procedure name`
Cause: The specified parameter for Atalla Security Module (ASM) was not found.
Remedy: Add the specified parameter for Atalla Security Module (ASM).

Number: 9124
Severity: 3
Text: `Invalid ASM parameter in Proc: <text>`
`<text> = secutils procedure name`
Cause: The specified for Atalla Security Module (ASM) was invalid.
Remedy: Correct the specified parameter for Atalla Security Module (ASM).

Number: 9125
Severity: 3
Text: `Missing VSM parameter in Proc: <text>`
`<text> = secutils procedure name`
Cause: The specified parameter for Visa Security Module (VSM) was not found.
Remedy: Add the specified parameter for Visa Security Module (VSM).

Number: 9126
Severity: 3
Text: `Invalid VSM parameter in Proc: <text>`
`<text> = secutils procedure name`
Cause: The specified for Visa Security Module (VSM) was invalid.
Remedy: Correct the specified parameter for VSM.

Number:	9127
Severity:	3
Text:	`DNT^STATUS^ flag is -1 indicating load failed for the Diebold  Number Table`
Cause:	The Diebold Number Table (DNT) load to security device is incompleted..
Remedy:	Retry load again and contact HELP24 if error still exists.

Number:	9128
Severity:	3
Text:	`DNT relative location on KEYA is invalid`
Cause:	Index number of Diebold Number Table was invalid.
Remedy:	Correct Diebold Number Table index .

Number:	9129
Severity:	3
Text:	`DNT load failure.  Security device message buffer content = <text>` `<text> = Visa security box’s response message`
Cause:	The Diebold Number Table (DNT) load to security device is incomplete..
Remedy:	Check response message that return from security device and correct. if error still exists contact HELP24.

Number:	9130
Severity:	3
Text:	`***** ALL SECURITY DEVICES FAILED INITIALIZATION *****`
Cause:	All security devices failed during initialisation.
Remedy:	Verify that all security devices are in valid state.

Number:	9131
Severity:	1
Text:	`<file> is still marked DOWN` `<file> = Security device name`
Cause:	Blocker of security device can not re-use after timer already expired.
Remedy:	Contact system manager.

Number: 9132
Severity: 1
Text: `Unknown SCM response code`
Cause: Response code return from security device was invalid.
Remedy: Verify that the security devices are in valid state.

Number: 9133
Severity: 3
Text: `Invalid message length to SCM`
Cause: The length of request message to security device was invalid.
Remedy: Check message format and message length that send to security device.

Number: 9134
Severity: 3
Text: `Requested function not supported`
Cause: The function type for security device was invalid.
Remedy: Correct function type in request message that was send to security device.

Number: 9135
Severity: 3
Text: `Invalid field context`
Cause: The field context in request message to security device was invalid.
Remedy: Correct message field that send to security device.

Number: 9136
Severity: 3
Text: `Invalid variant index`
Cause: The variant index number in request message to security device was invalid.
Remedy: Correct variant index number in message that send to security device.

Number: 9137
Severity: 3
Text: KM requested not held
Cause: `The KM index in request message to security device is not loaded in security device.`
Remedy: Either change the KM index in BASE24 or load KM in to security device.

Number: 9138
Severity: 1
Text: `KPV requested not held`
Cause: The KPV index in request message to security device is not loaded in security device.
Remedy: Either change the KPV index in BASE24 or load KPV in to security device.

Number: 9139
Severity: 1
Text: `Invalid MAC`
Cause: Invalid Message Authentication Coding (MAC) return from security device.
Remedy: Correct MAC keys.

Number: 9140
Severity: 1
Text: `Invalid padding in MAC residue`
Cause: Invalid pad character in Message Authentication Coding (MAC) in request message.
Remedy: Correct pad character in Message Authentication Coding (MAC).

Number: 9141
Severity: 3
Text: `Invalid PIN`
Cause: PIN verify failed return from security device.
Remedy: Use correct PIN.

Number: 9142
Severity: 1
Text: `Invalid PIN block content`
Cause: Invalid PIN block in request message to security device.
Remedy: Verify that PIN block is correct and make sure that security device is in a valid state.

Number: 9143
Severity: 1
Text: `Input buffer overflow`
Cause: Input buffer overflow in request message to security device.
Remedy: Correct message format and message length to security device.

Number: 9144
Severity: 3
Text: `Zero length PIN entered`
Cause: Invalid PIN length response return from security device.
Remedy: Correct PIN length field in message. It's should be greater than zero.

Number: 9145
Severity: 3
Text: `Termproof failure`
Cause: Proof-of-Endpoint for AS2805 terminal had fail.
Remedy: Verify that AS2805 terminal key is correct.

Number: 9146
Severity: 3
Text: `Public Key Pair not available`
Cause: Public Key Pair in request message to security device was not found.
Remedy: Correct Public Key.

Number: 9147
Severity: 3
Text: `KVC error`
Cause: KVC verify failed response return from security device.
Remedy: Verify that KVC. sent to security device is correct.

Number: 9148
Severity: 3
Text: `KPV validation error`
Cause: KPV validation error return from SCM.
Remedy: Verify that KVC. sent to security device is correct.

Number: 9149
Severity: 3
Text: `KPVV requested not held`
Cause: Response code return from SCM is KPVV requested not held.
Remedy: Check security setup.

Number: 9150
Severity: 1
Text: `KPV foreign`
Cause: Response code return from SCM is KPV foreign.
Remedy: Check security setup.

Number: 9151
Severity: 3
Text: `KPVV foreign`
Cause: Response code return from SCM is KPVV foreign.
Remedy: Check security setup.

Number: 9152
Severity: 3
Text: `Minimum PIN length requested not held`
Cause: Response code return from SCM is minimum PIN length not held.
Remedy: Check security setup.

Number: 9153
Severity: 3
Text: `Decimalisation Table requested not held`
Cause: Response code return from SCM is decimalisation table not held.
Remedy: Load decimalisation table in to security device or use an already loaded decimalisation table.

Number: 9154
Severity: 3
Text: `Item requested not held`
Cause: Response code return from SCM is item not held.
Remedy: Contact HELP24.

Number: 9155
Severity: 3
Text: `Signature validation failure`
Cause: Response code return from SCM is signature validation failure.
Remedy: Contact HELP24.

Number: 9156

Severity: 3

Text: `RSA cipher error`

Cause: RSA cipher error return from SCM

Remedy: Contact HELP24.

Number: 9158

Severity: 3

Text: `Fnameexpand failed for <secure-dev-name>`

Cause: Security device file name in Logical Network Configuration File (LCONF) is incorrect.

Remedy: Correct security device assign file name in Logical Network Configuration File (LCONF).

Number: 9159

Severity: 3

Text: `Unknown <secure-dev-typ>, default DES`

Cause: The security device type was invalid. The default security device type is DES.

Remedy: Correct security device type.

Number: 9163

Severity: 2

Text: `ANSI-ANSI PIN xlatn using different PAN strings not supported on Racal`

Cause: Racal device does not supported ANSI to ANSI pin translation using different PAN strings.

Remedy: Contact HELP24.

Number: 9165

Severity: 3

Text: `Diebold Number Table on the KEYA is invalid`

Cause: Diebold Number Table on the KEYA is invalid

Remedy: Contact HELP24.

Number:	9166
Severity:	1
Text:	`Invalid SECURE-DEV-MONITOR-THRESHOLD PARAM - DEFAULTING TO 0`
Cause:	Invalid SECURE-DEV-MONITOR-THRESHOLD PARAM ,set as default.
Remedy:	Check value of SECURE-DEV-MONITOR-THRESHOLD, it should be >= 0 and <= 32000.

Number:	9167
Severity:	1
Text:	`Invalid SECURE-DEV-CONF-TXT-PARAM - NSP DEFAULTS USED`
Cause:	SECURE-DEV-CONF-TXT-PARAM in LCONF file does not match with option^id in secutils program.
Remedy:	Check SECURE-DEV-CONF-TXT-PARAM in LCONF.

Number:	9168
Severity:	1
Text:	`ERROR response received from <file> during NSP INIT - device marked down` `<file> = Security device name`
Cause:	Invalid response from security device ,set device down.
Remedy:	Verify that security device is in a valid state.

Number:	9169
Severity:	1
Text:	`Secondary Security Device Failure - reverting to Primary Security Device`
Cause:	Failed response received from secondary security device and retry limit exceed, revert to primary security device.
Remedy:	Verify that the secondary security device is in a valid state.

Number:	9170
Severity:	1
Text:	`Invalid SECURE-DEV-CONF-CMD100 PARAM - REQUESTED CONFIG NOT (RE-)ENABLED`
Cause:	The SECURE-DEV-CONF-CMD100 param in Logical Network Configuration File (LCONF) was invalid.
Remedy:	Correct SECURE-DEV-CONF-CMD100 param in Logical Network Configuration File (LCONF).

Number: 9171

Severity: 1

Text: `ERROR response received from <file> during CMD 100 INIT - device marked down`

`<file> = Security device name`

Cause: Error response received during write command 100 to Network Security Process (NSP).

Remedy: Insurer correct CMD 100 string is entered in LCONF. This string must be obtained directly form Atalla and is specific to a NSP.

Number: 9172

Severity: 1

Text: `POSSIBLE CONFIG ERROR due to inconsistent security device subtypes`

Cause: An error occurred when all security device have different subtype.

Remedy: Verify that all security device should have the same subtype.

Number: 9173

Severity: 1

Text: `Invalid  SECURE-DEV-CONF-TXT PARAM - device marked down`

Cause: The SECURE-DEV-CONF-TXT param in Logical Network Configuration File (LCONF) was invalid ,set device down.

Remedy: Correct SECURE-DEV-CONF-TXT param in Logical Network Configuration File (LCONF).

Number: 9174

Severity: 1

Text: `Invalid SECURE-DEV-CONF-CMD100 PARAM - device marked down`

Cause: The invalid SECURE-DEV-CONF-CMD100 param return from security device, set device marked down.

Remedy: Correct SECURE-DEV-CONF-CMD100 param in Logical Network Configuration File (LCONF).

Number:	9175
Severity:	3
Text:	`Missing SECURE-DEV-TYPE PARAM`
Cause:	Security device type in Logical Network Configuration File (LCONF) file does not exist.
Remedy:	Add SECURE-DEV-TYPE in Logical Network Configuration File (LCONF) file.

Number:	9180
Severity:	3
Text:	`ESM response BCC check failed`
Cause:	BCC check failed return from ESM. The BCC is being calculated incorrectly.
Remedy:	Contact HSLP24.

Number:	9181
Severity:	1
Text:	`Invalid <secure-dev-reinstate-time> parameter - defaulting to 0`
Cause:	The reinstate time of security device in Logical Network Configuration File (LCONF) was invalid. The default reinstate time is 0
Remedy:	Correct secure-dev-reinstate-time param in Logical Network Configuration File (LCONF).

Number:	9182
Severity:	3
Text:	`All security devices in a down state`
Cause:	All security devices have failed response to process.
Remedy:	Contact system manager.

Number:	9183
Severity:	3
Text:	`Diebold PIN verification specified for ERACOM !!!. Not possible`
Cause:	Diebold PIN verification method for ERACOM is not supported.
Remedy:	Verify that PIN verification method for ERACOM is correct. If Diebold PIN verification is required, contact HELP24.

Number:	9184
Severity:	1
Text:	`invalid <secure-dev-access-type> parameter - defaulting to 0`
Cause:	The security access type in Logical Network Configuration File (LCONF) was invalid. The default access type is 0.
Remedy:	Correct the security access type in Logical Network Configuration File (LCONF).

Number:	9185
Severity:	3

Text:	`Security device <file> down.  Reinstated at <text>`
	`<file> = Security device name`
	`<text> = timer for reinstate state.`

Cause:	Security device had failed response to process and re-open again when re-open timer reached.
Remedy:	Verify that the security device is in a valid state.

Number:	9186
Severity:	1
Text:	`Number of security devices still up = <text>` `<text> = Number of security devices.`
Cause:	Display number of good security devices for process.
Remedy:	Depend on number of security devices that still up.

Number:	9187
Severity:	3

Text:	`Attempt to reinstate device <file> failed - <error>`
	`<file> = Security device name`
	`<error> = Error number`

Cause:	Can not re-open security device.
Remedy:	Contact system manager.

Number:	9188
Severity:	0
Text:	`Device <file> reinstated successfully` `<file> = Security device name.`
Cause:	Process can re-open security device after security device had fail.
Remedy:	No action required.

Number: 9189

Severity: 1

Text: `Invalid <secure-dev-trn-retry^cnt parameter - defaulting to 5`

Cause: The security retry count in Logical Network Configuration File (LCONF) was invalid The default retry count is 5.

Remedy: Correct the security retry count in Logical Network Configuration File (LCONF).

Number: 9190

Severity: 1

Text: `<secure-dev-trn-retry^cnt> > 100, ridiculous. Defaulting to 5`

Cause: The security retry count in Logical Network Configuration File (LCONF) can not greater than 100. The default retry count is 5.

Remedy: Correct the security retry count in Logical Network Configuration File (LCONF).

Number: 9192

Severity: 1

Text: `Security Device Type <file>, fails = <text>, State = <text>`

`<file> = Security device name`

`<text> = Number of failure`

`<text> = Status of security device`

Cause: Display security device status information for each device.

Remedy: Depend on number of failure and status of security device.

Number: 9193

Severity: 1

Text: `Security Device Type = <text1>, next = <file>, Operational = <text1>`

`<text1> = Security device type`

`<file> = Next security device name`

`<text2> = Number of device operation`

Cause: Display general status message on the security devices

Remedy: No action required.

Number: 9194

Severity: 1

Text: `Invalid <secure-dev-comms-type> parameter - defaulting to 0`

Cause: Value of secure-dev-comms-type in LCONF file is incorrect, set as default.

Remedy: Update secure-dev-comms-type in LCONF file.

Number: 9195

Severity: 3

Text: `Blocker process marked down.  Closed <file>`
`<file> = Security device file name`

Cause: Fail response received from blocker process, set blocker process down.

Remedy:

Number: 9196

Severity: 3

Text: `Blocker process <file> marked UP`
`<file> = Security device name`

Cause: Good response return from blocker, set blocker process up.

Remedy: No action required.

Number: 9197

Severity: 3

Text: `All BLOCKER processes are marked DOWN`

Cause: The device error response return while calling send^message to security device and marked all blocker down for this security device.

Remedy: Restart BLOCKER processes.

Number: 9198

Severity: 3

Text: `Invalid <secure-dev-type> expecting 'SCM'`

Cause: The security device type in Logical Network Configuration File (LCONF) mismatch.

Remedy: Correct security device type in Logical Network Configuration File (LCONF).

Number:	9199
Severity:	3
Text:	`Unable to OPEN any BLOCKER process`
Cause:	Can not open all blocker process.
Remedy:	Check blocker process.

Number:	9200
Severity:	3
Text:	`**** SECUTIL dummy proc <text> called ****` `<text> = secutils procedure name`
Cause:	Attempt to call dummy proc.
Remedy:	Contact HELP24.

Number:	000
Severity:	3
Text:	`PINI^LOG^MSG was called with an unknown msgnum of <num>` `<num> = Error message number`
Cause:	Programatic error.
Remedy:	Contact HELP24.

Applied Communications, Inc.